



Hall Booking and Management System

S.F SAFIYA

KUR/IT/2324/F/0080

Hall Booking and Management System

PROJECT REPORT PRESENTED BY

S.F SAFIYA

KUR/IT/2324/F/0080

HIGHER NATIONAL DIPLOMA IN INFORMATION TECHNOLOGY
Of the

ADVANCED TECHNOLOGICAL INSTITUTE – KURUNEGALA
SRI LANKA INSTITUTE OF ADVANCED TECHNOLOGICAL EDUCATION (SLIATE)

DECLARATION

I hereby declare that this project report, Hall Booking & Management System, is entirely my own work and has been carried out under the supervision of Ms. P.G.R.N.J. Gamlath. This project has not been submitted, either in whole or in part, for any other academic award or qualification.

I further declare that all sources of information, ideas, and materials obtained from books, journals, websites, and other references have been appropriately acknowledged and cited in this report. Any assistance received during the development of this project has also been duly acknowledged.

This report is submitted in partial fulfillment of the requirements for the Higher National Diploma in Information Technology (HNDIT) at the Sri Lanka Institute of Advanced Technological Education (SLIATE). No part of this report may be reproduced, stored, or transmitted in any form or by any means without the prior written permission of the author.

Signature of Candidate:

Name: S.F. Fathima Safiya

Registration Number:

Date:

Certified by:

Supervisor Name: Ms. P.G.R.N.J. Gamlath

Signature:

Date:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Ms. P.G.R.N.J. Gamlath, for the continuous guidance, patience and constructive feedback that were given to me throughout the development of this project. Their advice on structuring the booking workflow, reviewing each milestone and questioning my design decisions helped me stay focused on building a system that satisfies both academic requirements and real business needs, rather than a system that only looks good on the surface.

I am also thankful to the management and staff of Camilla Banquet Hotel, who patiently explained how their booking and payment processes are currently handled using physical ledgers, phone calls and manual receipts. The time they spent describing their daily operations, their common problems with double bookings and their method of collecting advance payments became the foundation for almost every business rule that is implemented inside HallCamilla today. Without this information, the system would have been built on assumptions instead of real operational needs.

My appreciation also goes to the technical mentors and fellow students who reviewed early versions of the system architecture with me, particularly the parts related to Firebase Authentication, Cloud Firestore data modelling and the integration of the PayHere Sandbox payment gateway. Their comments helped me identify weaknesses in my original design, such as how payment records should be linked back to bookings, and encouraged me to correct these issues before the final submission.

Finally, I would like to thank my family and friends for their patience, understanding and encouragement during the long hours that were spent designing, coding, testing and documenting this project. Their support made it possible for me to complete this work to the standard required for this final report.

ABSTRACT

The hospitality and event management industry is increasingly dependent on digital tools that can manage customer bookings, payments and communication in a fast and reliable manner. Many small and medium sized banquet halls, however, still rely on manual, paper based methods to record reservations, calculate advance payments and track the balance owed by each customer. As the number of events handled by a venue grows, this manual approach becomes difficult to sustain and commonly results in double bookings for the same date, lost or damaged paper records, and a poor communication experience for customers who have no way of checking their booking status without calling the hotel directly.

HallCamilla was designed and developed to resolve these problems for Camilla Banquet Hotel by replacing the manual ledger system with a centralised, cloud hosted web application. The system automates the complete reservation lifecycle, starting from the moment a customer submits a booking request, through administrative approval, advance and final payment collection, and finally the completion of the event. It was built using React together with TypeScript for the frontend user interface, Tailwind CSS for styling, Firebase Authentication and Cloud Firestore for backend data storage and user management, EmailJS for automated email communication, and the PayHere Sandbox for processing simulated online card payments.

The application implements a role based authentication system that separates the customer facing portal from the administrative dashboard, an interactive multi step booking workflow that checks hall availability in real time before a request is accepted, and a hybrid payment system that allows customers to pay a mandatory twenty percent advance and the remaining eighty percent balance either securely online through PayHere or in person as an offline cash payment recorded by the administrator. A comprehensive administrative dashboard was also built to give hotel staff full visibility over pending requests, confirmed bookings, payment records and overall revenue.

The final outcome of this project is a fully functional, cloud hosted booking and management system that removes the need for physical paperwork, mathematically prevents double bookings by enforcing availability rules at the database level, keeps both customers and administrators informed through real time notifications, and gives Camilla Banquet Hotel a modern digital identity that reflects the premium nature of the venue itself.

TABLE OF CONTENTS

DECLARATION	3
ACKNOWLEDGEMENT	4
ABSTRACT.....	5
TABLE OF CONTENTS.....	6
TABLE OF FIGURES.....	8
LIST OF TABLES.....	10
LIST OF ABBREVIATIONS.....	11
CHAPTER 1 : INTRODUCTION	13
1.1 Purpose.....	13
1.2 Goals & Vision	15
CHAPTER 2 : REQUIREMENT SPECIFICATION	17
2.1 Requirement Gathering Process.....	17
2.2 System Functions	17
2.3 Software Requirements.....	19
2.4 Hardware Requirements.....	20
CHAPTER 3 : DESIGN.....	21
3.1 Front-End Design (User Interfaces).....	21
3.2 Database Design.....	52
3.3 Architectural Design	58
CHAPTER 4 : IMPLEMENTATION	65
4.1 Development Approach	65
4.2 Implementation of Key Modules	65
4.3 Challenges Faced	67
CHAPTER 5 : TESTING.....	68
5.1 Testing Approach.....	68
5.2 Test Cases	69
5.3 User Acceptance Testing Summary.....	70
CHAPTER 6 : RESULTS AND EVALUATION	71
6.1 Achievement of Objectives.....	71
6.2 Limitations	71
CHAPTER 7 : CONCLUSION AND FUTURE WORK.....	73
7.1 Conclusion	73
7.2 Future Work.....	74
APPENDICES	75

Appendix A: List of Abbreviations.....	75
REFERENCES	76

TABLE OF FIGURES

Figure 3. 1: Home Page	22
Figure 3. 2: Hall Catalog Page.....	23
Figure 3. 3 : Hall Details Page	24
Figure 3. 4 : Budget Calculator Page	25
Figure 3. 5 : Sign In Page.....	26
Figure 3. 6 : Registration Page.....	27
Figure 3. 7 : Forgot Password Page	28
Figure 3. 8 : Customer Dashboard Page	29
Figure 3. 9 : Customer Profile Page.....	30
Figure 3. 10 : Wishlist Page.....	31
Figure 3. 11 : Event Customization Page.....	32
Figure 3. 12 : Booking Review & Checkout Page.....	33
Figure 3. 13 : Payment Page	34
Figure 3. 14 : Customer Notifications Page.....	35
Figure 3. 15 : Automated Email Notification	36
Figure 3. 16 : Customer Review Page	37
Figure 3. 17 : Admin Dashboard Page.....	38
Figure 3. 18 : Booking Management Page.....	39
Figure 3. 19 : Hall Management Page	40
Figure 3. 20 : Package Management Page	41
Figure 3. 21 : Booking Cancellation Page	42
Figure 3. 22 : Review Management Page	43
Figure 3. 23 : Payment Management Page	44
Figure 3. 24 : Reports & Analytics Page	45
Figure 3. 25 : Customer Management Page.....	46
Figure 3. 26 : Event Types Management Page	47
Figure 3. 27 : Hall Maintenance Page.....	48
Figure 3. 28 : Availability Management Page	49
Figure 3. 29 : Notifications Management Page	50
Figure 3. 30 : Admin Settings Page	51
Figure 3. 31 : Entity Relationship Diagram.....	55

Figure 3. 32 : System Architecture Diagram	59
Figure 3. 33 : Use Case Diagram	60
Figure 3. 34 : Activity Diagram	61
Figure 3. 35 : Sequence Diagram.....	63

LIST OF TABLES

Table 3. 1 : Firestore Collection Structure and Relationship Mapping	53
Table 5 . 1 : Representative Test Cases	69
Table A . 1 : Abbreviations Used in This Report	75

LIST OF ABBREVIATIONS

The following abbreviations are used throughout this report.

Abbreviation	Meaning
AC	Air Conditioning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
EmailJS	EmailJS Email Service
ERD	Entity Relationship Diagram
FAQ	Frequently Asked Questions
Firebase	Google Firebase Platform
Firebase Auth	Firebase Authentication
Firestore	Firebase Cloud Firestore
HNDIT	Higher National Diploma in Information Technology
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LKR	Sri Lankan Rupee
NoSQL	Non-Relational Database

Abbreviation	Meaning
PDF	Portable Document Format
REST	Representational State Transfer
SaaS	Software as a Service
SDK	Software Development Kit
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQA	Software Quality Assurance
SRS	Software Requirements Specification
TS	TypeScript
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
Vite	Vite Frontend Build Tool
React	React JavaScript Library
Tailwind CSS	Tailwind CSS Framework
PayHere	PayHere Online Payment Gateway

CHAPTER 1 : INTRODUCTION

1.1 Purpose

The primary purpose of the HallCamilla project is to digitally transform the hall booking and management process of Camilla Banquet Hotel by replacing the traditional manual reservation system with a centralized web-based application. The system enables administrators to manage hall reservations, customer information, payments, and booking schedules efficiently while providing customers with real-time hall availability, transparent pricing, and an online booking experience. By storing all booking and payment records in Firebase Cloud Firestore, the system ensures data accuracy, security, and real-time accessibility.

In addition to addressing a real-world business problem, this project also fulfills the academic requirements of the Higher National Diploma in Information Technology (HNDIT) programme. It demonstrates practical knowledge of modern web application development through the use of React, TypeScript, Firebase Authentication, Cloud Firestore, Tailwind CSS, EmailJS, and the PayHere Sandbox payment gateway. The project showcases the ability to independently design, develop, test, and deploy a complete full-stack software solution that meets both functional and non-functional requirements.

System Overview

HallCamilla is a web-based single-page application consisting of two main modules: the Customer Portal and the Administrator Dashboard. Although both modules share the same application and Cloud Firestore database, the available features are determined by the authenticated user's role. This role-based structure provides customers with a simple booking experience while enabling administrators to efficiently manage the hotel's booking operations.

The Customer Portal allows users to browse available halls and event packages, check hall availability, calculate event costs, customize bookings with additional services, and submit reservation requests. Customers can also manage their profiles, track booking status, make online payments through the PayHere payment gateway, receive notifications, maintain a wishlist, and submit reviews after completed events.

The Administrator Dashboard provides tools for managing halls, packages, bookings, customers, payments, event types, maintenance schedules, reviews, reports, notifications, and system settings. Administrators can approve or reject bookings, update booking and payment

statuses, manage venue availability, and monitor business performance through dashboards and reports.

All application data is stored in Firebase Cloud Firestore, while Firebase Authentication provides secure user authentication and role-based access control. Firestore's real-time synchronization ensures that updates made by customers or administrators are immediately reflected throughout the system without requiring manual page refreshes.

Problem Statement

Camilla Banquet Hotel, like many similar hospitality businesses, has traditionally relied on a fully manual approach for managing hall reservations. New booking enquiries were recorded manually in reservation ledgers, while advance payments and final balances were documented using physical receipts. Although this approach may be sufficient for a small number of events, it becomes increasingly difficult to manage as booking volume grows. These limitations created the primary motivation for developing the HallCamilla system.

The first and most critical issue identified was the risk of double bookings. Since staff members had to manually check physical records whenever a new reservation request was received, there was a possibility of overlooking existing reservations and accidentally accepting multiple events for the same hall and date. Such conflicts, especially when discovered close to the scheduled event date, could negatively impact customer satisfaction, damage the hotel's reputation, and result in financial losses.

The second major issue was related to payment and financial management. Advance payments and remaining balances were calculated manually, which increased the possibility of calculation errors and required additional administrative effort. Furthermore, the absence of a centralized digital record made it difficult for management to obtain accurate and up-to-date information about collected revenue, outstanding payments, and overall booking performance.

The third issue was the lack of transparency and accessibility for customers. Without a digital booking platform, customers were required to contact the hotel through phone calls or physical visits to obtain information about their reservation status or payment history. This created inconvenience for customers while increasing the workload of hotel staff who had to repeatedly respond to similar booking-related enquiries.

HallCamilla was developed to address these challenges through a centralized digital booking and management solution. The risk of double bookings is minimized through automated availability validation, where the booking service checks Cloud Firestore for existing active reservations before allowing a new booking request to proceed. Financial calculations, including the 20% advance payment and 80% remaining balance, are automatically calculated by the system and recorded within the payment management process, reducing manual calculation errors.

Furthermore, customer transparency is improved through the customer dashboard, real-time booking updates, automated email notifications, and in-app notifications. These features allow customers to access accurate information about their booking status and payment progress without needing to directly contact hotel staff, while also improving the efficiency of hotel operations.

1.2 Goals & Vision

The primary goal that guided the development of HallCamilla was to create a reliable, secure, and user-friendly web application capable of managing the complete lifecycle of a banquet hall reservation, from the initial booking request until the final completion of the event. Reliability was achieved by implementing robust availability validation mechanisms that prevent conflicting reservations, security was ensured through role-based access control that restricts administrative operations to authorized users, and usability was addressed by designing an interface that can be effectively used by both administrators and customers with different levels of technical experience.

By the completion of the development period, several key objectives were successfully achieved. A role-based authentication and authorization system was implemented using Firebase Authentication together with a Firestore users collection that stores user roles and access permissions. A hybrid payment management system was also developed, supporting online card transactions through the PayHere Sandbox gateway and manual recording of offline cash payments by administrators. In addition, a real-time database architecture was successfully implemented using Cloud Firestore, allowing important system changes, such as administrator approval of booking requests, to be immediately reflected on customer dashboards without requiring manual page refreshes.

The long-term vision for HallCamilla is to expand the system into a scalable multi-tenant platform capable of supporting multiple banquet properties through a single centralized solution. Future improvements may include vendor and supplier management, customizable catering and service selections during the booking process, and integrated staff scheduling features to support better event resource planning.

Even in its current single-hotel implementation, HallCamilla provides significant operational benefits by reducing manual administrative workload, improving accuracy in financial and booking management, providing real-time visibility of reservation activities, and enhancing the customer experience through a modern digital booking platform. The system transforms the traditional phone-based reservation and paper-based record approach into a more efficient, transparent, and professional online solution.

CHAPTER 2 : REQUIREMENT SPECIFICATION

2.1 Requirement Gathering Process

The requirements for HallCamilla were not taken from a generic template of hotel booking features. Instead, they were identified through a combination of business analysis and direct observation of how a typical banquet hall actually operates on a daily basis. The first step was to study the standard operational procedure that a venue like Camilla Banquet Hotel follows when converting a simple customer enquiry into a confirmed, paid reservation, which includes checking hall availability, discussing package pricing, collecting an advance payment to secure the date and eventually collecting the remaining balance before or on the day of the event.

Alongside this business analysis, particular attention was paid to observing the pain points of the existing manual workflow, especially the physical booking ledger and the handwritten cash receipts that were used to track payments. These observations made it clear that the two biggest sources of frustration for the hotel were the risk of double booking a hall and the difficulty of keeping an accurate running total of how much money had actually been collected for each event. Once these pain points were understood, a functional analysis was carried out to translate each manual step into a corresponding digital action, for example transforming the informal decision of a manager to approve a request into a formal status change from Pending to Approved inside the system, and transforming the manual calculation of a deposit into an automatic twenty percent advance calculation performed by the payment service.

This grounded, observation based approach to requirement gathering was deliberately chosen over simply listing generic hotel software features, because the real goal of the project was to build something that hotel staff would find genuinely useful and intuitive in their actual daily routine, rather than a system that looks impressive on paper but does not match how the business really works.

2.2 System Functions

HallCamilla is made up of five main functional modules that work together to support both the customer and the administrator sides of the system. Each module was implemented as a combination of a Firestore collection for data storage, one or more service files that contain the business logic, and the React components that the user actually interacts with.

The Authentication Module allows a new user to register an account and allows both customers and administrators to securely log in and maintain their session while using the application. This module exists because sensitive customer information, such as contact details and event dates, along with the administrative controls that manage the whole hotel, must be protected from unauthorised access. It is implemented using the Firebase Authentication service for verifying login credentials, a users collection in Firestore that stores each account's profile details and assigned role, and a global AuthContext in React that keeps the current user's session information available throughout the entire application without needing to pass it manually between components.

The Booking Management module, which is used from the customer side, allows a client to select an event date, specify the number of guests attending and choose a service package before submitting a formal reservation request. This feature gives the hotel a structured, digital record of every potential booking instead of an informal phone enquiry, and it is supported by the bookings collection in Firestore, the bookingService.ts file that contains the logic for checking availability and writing new booking documents, and the BookingForm.tsx component that guides the customer through the request step by step.

The Booking Administration module is used from the administrator side and allows hotel staff to review every incoming request and decide whether to approve or reject it. When an administrator approves a request, the system automatically sends a confirmation email to the customer and updates the booking's status in the database, which also has the effect of preventing any other customer from later claiming the exact same date for the same hall. This module relies mainly on the AdminBookings.tsx interface for displaying and acting on requests, together with the emailNotificationService.ts file that is responsible for dispatching the relevant EmailJS messages.

The Hybrid Payment Gateway is one of the most important modules in the system because it is directly responsible for the hotel's cash flow. Customers use this feature to pay a mandatory twenty percent advance through the PayHere Sandbox once their booking has been approved, which upgrades the booking's status to Confirmed, and later they can pay the remaining eighty percent balance either online through the same gateway or in person at the hotel, in which case an administrator manually records the cash payment through the admin payments dashboard. This module is built around the payments collection in Firestore, the paymentService.ts file that calculates the correct advance and balance amounts, the Payment.tsx page that manages

the checkout flow, and the `PayHereButton.tsx` component that actually initialises the PayHere widget.

Finally, the Automated Notifications module dispatches both real time in app alerts and emails whenever an important event occurs in the system, such as a new booking being submitted, a booking being approved or a payment being successfully processed. This keeps everyone involved informed automatically, without any manual intervention from staff, and it is powered by the notifications collection in Firestore, the `NotificationBell.tsx` component that displays a live counter of unread alerts, and the `useNotifications` custom hook that subscribes to the relevant Firestore documents in real time.

2.3 Software Requirements

HallCamilla was developed using a modern, entirely JavaScript based technology stack that was chosen to balance performance, developer productivity and long term maintainability. React was selected as the core library for building the user interface because its component based architecture made it possible to build small, reusable pieces of the interface, such as buttons, modals and form fields, and reuse them consistently across both the customer and administrator sides of the application. TypeScript was used instead of plain JavaScript throughout the project because its static typing catches many simple mistakes, such as passing the wrong data shape into a function, before the code is even run, which significantly reduced the number of runtime errors encountered during development.

Vite was chosen as the build tool for the project mainly because of its fast hot module replacement during development, which allowed changes made to a component to appear in the browser almost instantly without a full page reload, and because of the optimised, smaller production bundles it generates when the application is finally built for deployment. Tailwind CSS was used for all of the styling in the project, since its utility first approach made it possible to design the premium looking Luxury Emerald and Gold interface quickly, without needing to maintain a large separate collection of custom CSS files for every component.

On the backend side, Firebase Authentication and Cloud Firestore were selected together as a complete backend as a service solution. This decision removed the need to write, host and maintain a separate custom backend server, since Firebase already provides secure user authentication and a real time NoSQL database out of the box. React Router was used to manage navigation between the public marketing pages, the customer dashboard pages and the

administrator pages, allowing the whole project to function as a single page application without full page reloads when the user moves between sections. The PayHere Sandbox was integrated specifically to simulate a realistic, locally relevant online payment experience for Sri Lankan customers. Supporting all of this development work, Node.js and npm were used to manage the project's package dependencies, Git was used for version control throughout the development period, and Visual Studio Code was used as the primary development environment.

2.4 Hardware Requirements

Because HallCamilla is a cloud hosted, browser based application, its hardware requirements are considerably lighter than those of a traditional desktop software product, but a few practical requirements still apply on both the development and end user sides.

For development purposes, the machine used to build and test HallCamilla needed at least a modern multi core processor, such as an Intel Core i5 or an equivalent AMD Ryzen 5 chip, together with a minimum of eight gigabytes of RAM and a solid state drive with at least ten gigabytes of free storage space. These specifications were necessary to comfortably run the Node.js runtime, the Vite development server and a code editor such as Visual Studio Code at the same time without noticeable slowdown.

For the people who actually use the finished application, whether they are customers making a booking or administrators managing the hotel, the hardware requirement is simply any modern device capable of running an up to date web browser such as Chrome, Firefox, Safari or Edge. This includes ordinary desktop computers, laptops, tablets and smartphones, which means that a customer could realistically check their booking status or make a payment from their mobile phone while an administrator manages the same reservation from a desktop computer at the hotel's front desk. Because the entire system is hosted in the cloud through Firebase, there are no local server machines that the hotel itself needs to purchase, configure or maintain in order to run HallCamilla.

CHAPTER 3 : DESIGN

3.1 Front-End Design (User Interfaces)

The front end of HallCamilla was designed to reflect the image of a premium hospitality brand rather than a conventional booking system, aligning with the hotel's focus on luxury weddings, birthday celebrations, and corporate events. The user interface follows a Luxury Emerald and Gold theme, where deep emerald green represents elegance, trust, and stability, while gold accents highlight primary actions, pricing information, and important notifications. This colour scheme guides users naturally through the booking process while reinforcing the hotel's brand identity.

Typography combines a classic serif font for headings with a modern sans-serif font for body text and form fields. This approach provides a balance between a sophisticated appearance and readability, ensuring that the interface remains visually appealing while supporting a smooth user experience.

Navigation is provided through a responsive top navigation bar that adapts to both desktop and mobile devices. The system clearly separates customer-facing pages from administrative pages to provide interfaces tailored to each user role. Customer pages emphasize a simple, step-by-step booking workflow, allowing users to easily browse halls, customize events, complete payments, and monitor reservations. In contrast, the administrator pages prioritize operational efficiency through dashboards, data tables, analytical charts, and management tools that support booking administration, payment tracking, reporting, and system maintenance.

The following sections present the implemented user interfaces of the HallCamilla system in the sequence that customers and administrators interact with them during normal system operation.

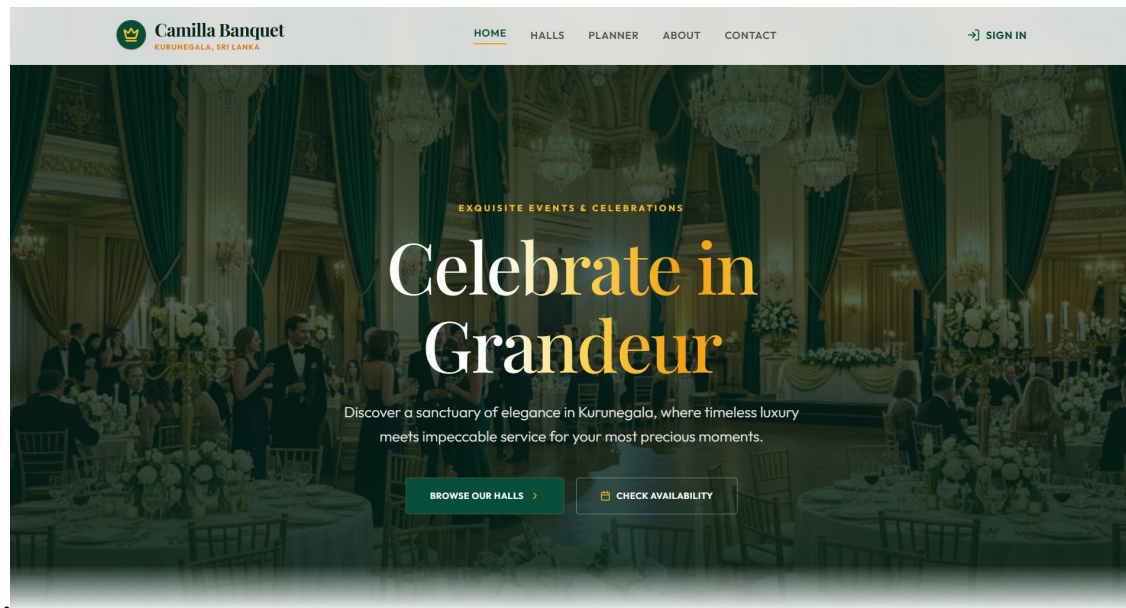


Figure 3. 1: Home Page

The Home Page is the main entry point of the HallCamilla system and provides users with an attractive introduction to the hotel services. It uses a luxury emerald green and gold theme that represents the hotel branding. The page includes a hero banner with clear booking call-to-actions, featured halls, event packages, customer reviews, and important hotel information.

Hall and package details are dynamically retrieved from **Firebase Firestore**, allowing administrator updates such as pricing, images, and descriptions to be reflected automatically. Navigation links provide quick access to the Hall Catalog, Budget Calculator, Login, and Registration pages.

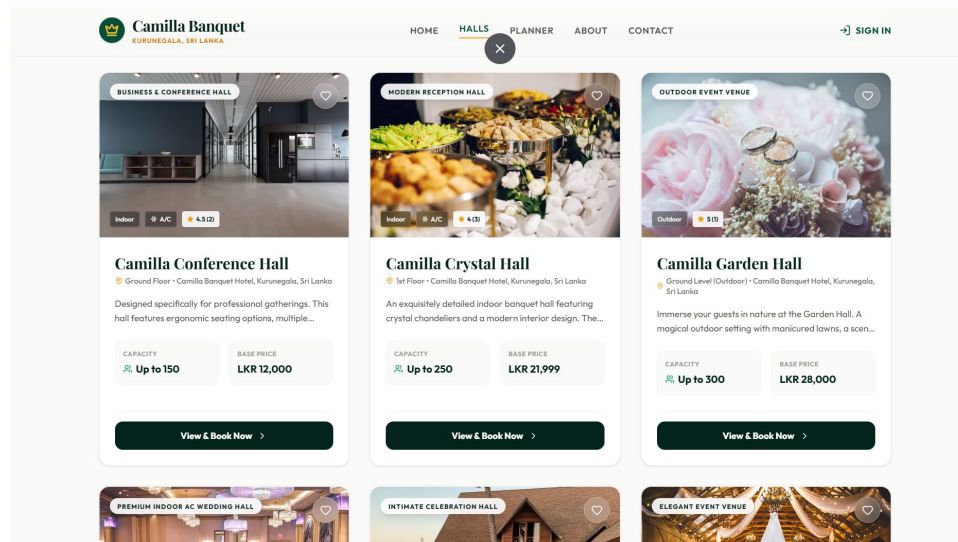


Figure 3. 2: Hall Catalog Page

The Hall Catalog Interface allows users to explore available halls and event packages through an organized card-based layout. Each card displays important details such as hall name, images, starting price, guest capacity, location, and a brief description.

The interface provides search and filtering options to help users find suitable halls based on their event requirements. Hall information is retrieved from Firebase Firestore, ensuring that any updates made by administrators are displayed in real time.

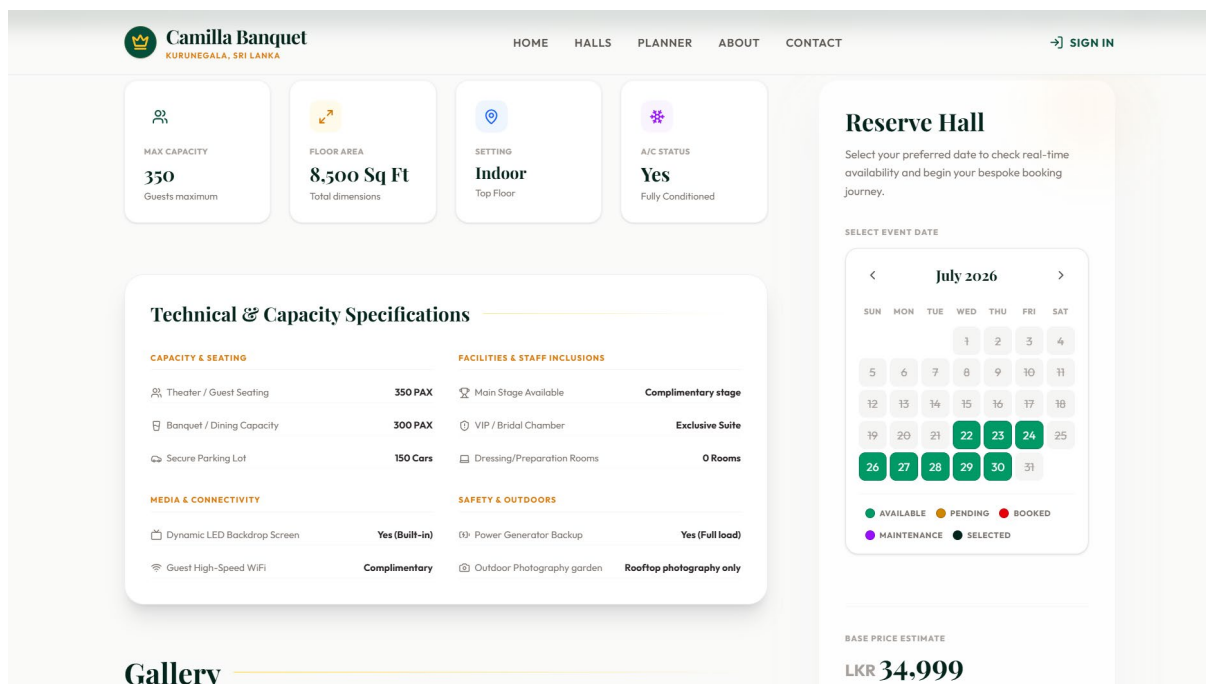
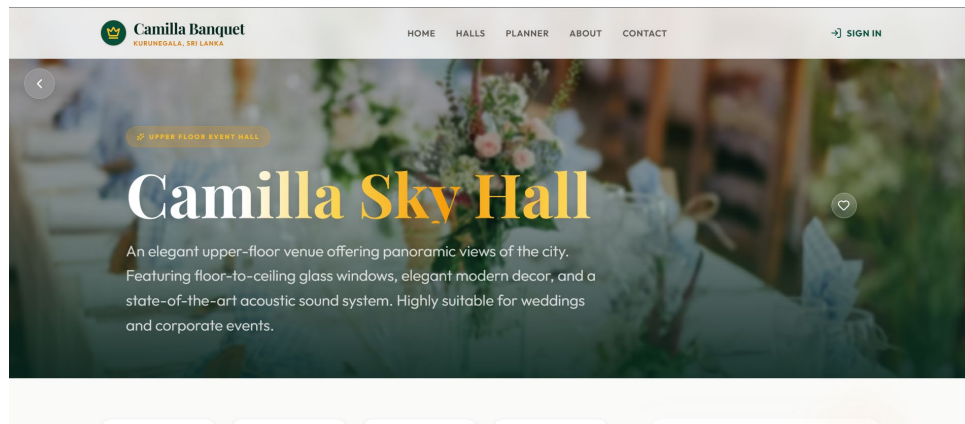


Figure 3. 3 : Hall Details Page

The Hall Details Interface provides detailed information about a selected hall, including image galleries, descriptions, pricing, guest capacity, amenities, and available services.

Users can save preferred halls using the Wishlist feature or proceed with reservations through the Book Now option. The system retrieves hall details dynamically from Firestore using the hall ID provided through the URL.

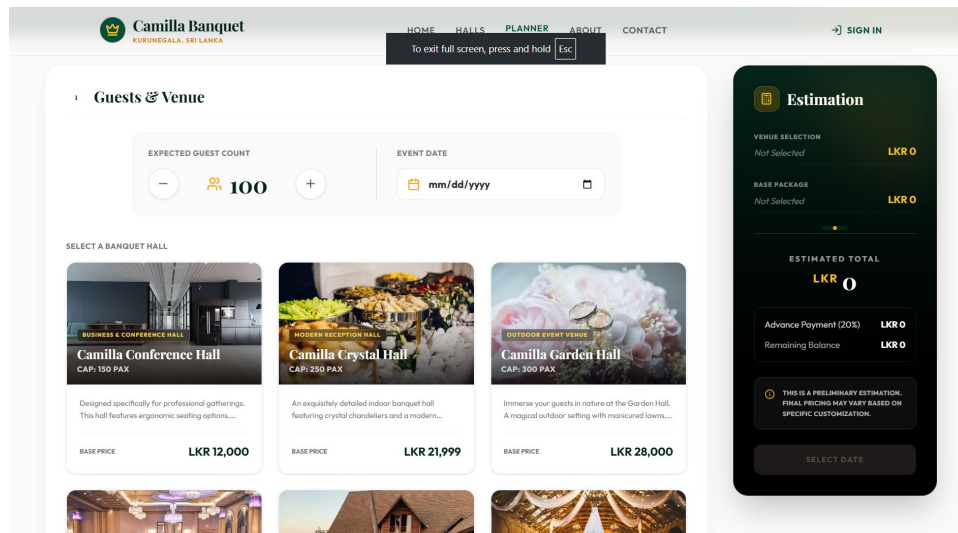


Figure 3. 4 : Budget Calculator Page

The Budget Calculator Interface helps customers estimate their expected event cost before booking. Users can select a hall package, enter the expected guest count, and choose additional services such as photography, catering, and DJ services.

The system performs real-time calculations by combining package prices, guest numbers, and selected service charges. This provides customers with a clear cost estimation and reduces uncertainty about the total booking amount.

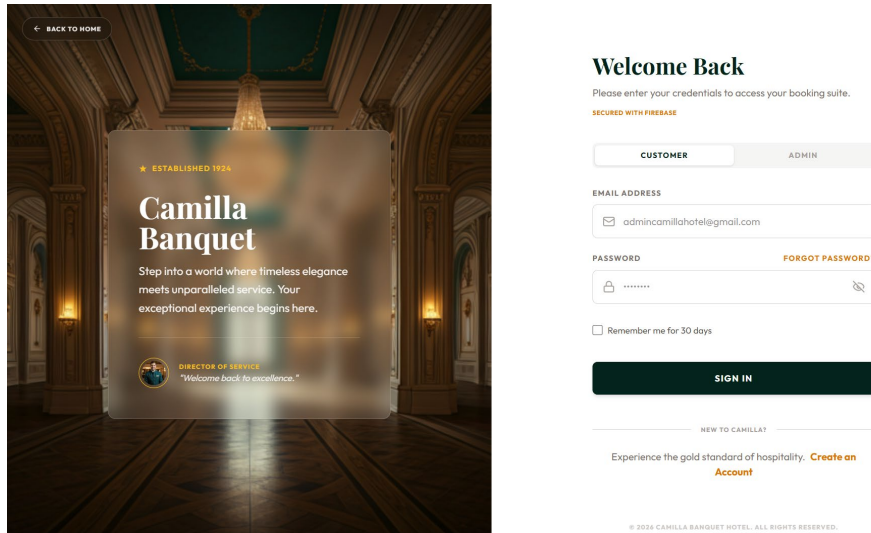


Figure 3. 5 : Sign In Page

The Login Interface provides secure access to the system for both customers and administrators. Users can enter their email and password, with additional options for registration and password recovery.

The system verifies credentials through Firebase Authentication. After successful login, users are redirected to their relevant dashboard based on their assigned role, either Customer or Admin.

← BACK TO HOME

★ ESTABLISHED 1924

Camilla Banquet

Join an exclusive community where every event is a masterpiece. Experience the pinnacle of luxury.

Create Account

Step into a world of curated elegance and seamless hospitality.

SECURED WITH FIREBASE

FULL NAME

Johnathan Doe

EMAIL ADDRESS

john@example.com

PHONE NUMBER

+94 77 123 4567

PASSWORD

CONFIRM

☐ I agree to the [Terms of Service](#) and [Privacy Policy](#).

CREATE ACCOUNT

ALREADY HAVE AN ACCOUNT?

Welcome back to excellence. [Sign In](#)

© 2026 CAMILLA BANQUET HOTEL. ALL RIGHTS RESERVED.

Figure 3. 6 : Registration Page

The Register Interface allows new customers to create an account by entering details such as full name, email address, phone number, and password.

The system validates user inputs before creating the account through Firebase Authentication. After successful registration, customer details are stored in the Firestore users collection with the default Customer role, enabling access to booking features.

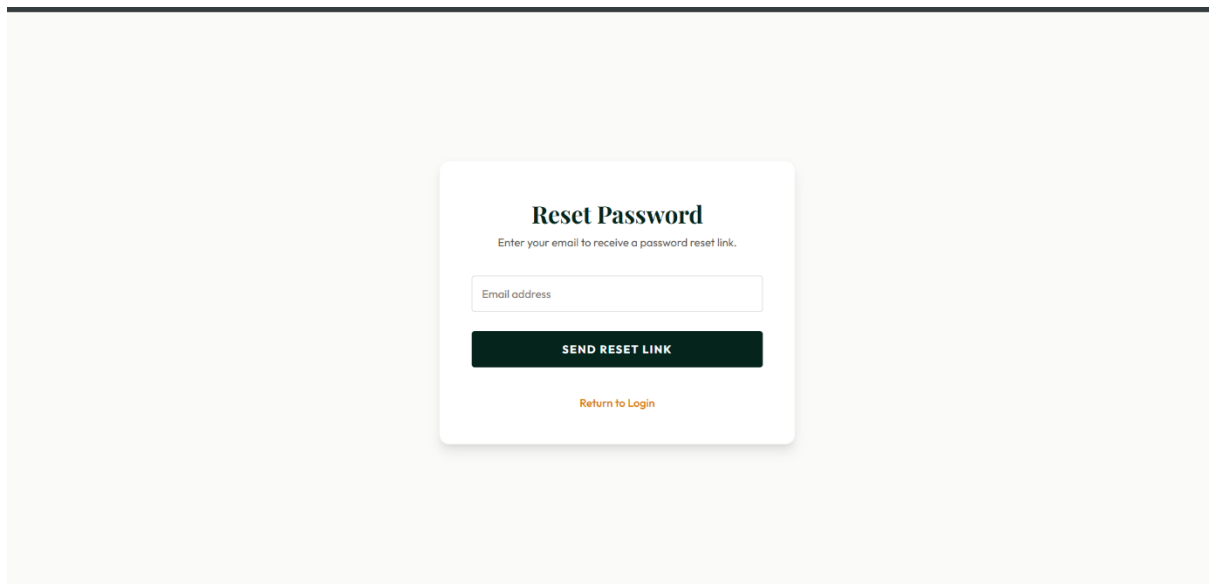


Figure 3. 7 : Forgot Password Page

The Forgot Password Interface provides a secure method for users to recover access to their accounts. Users only need to enter their registered email address to request a password reset.

The system uses the Firebase Authentication password recovery service to generate and send a secure reset link to the user's email, allowing them to update their password without administrator involvement.

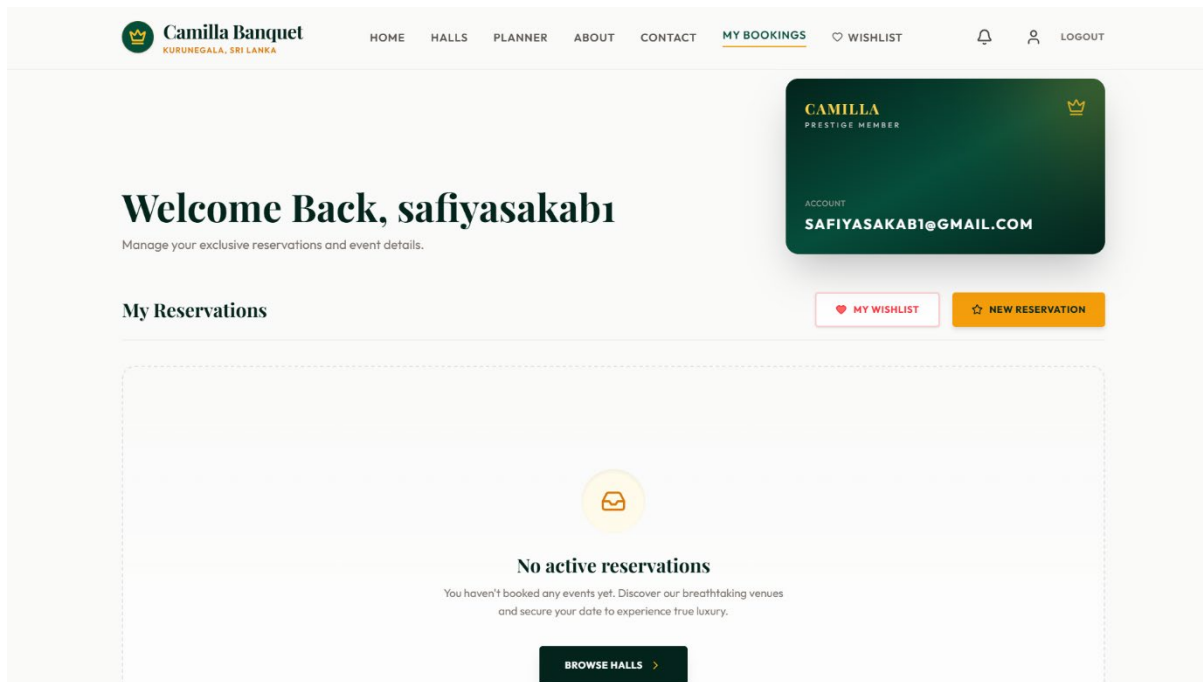


Figure 3. 8 : Customer Dashboard Page

The Customer Dashboard acts as a personal management area where users can monitor their booking activities. It displays summary information such as total bookings, upcoming events, and recent booking history.

Booking details including event dates, hall information, payment status, and reservation status are retrieved from Firestore. User-based filtering ensures that customers can only access their own booking information.

The screenshot displays the 'Personal Information' section of a user profile on the Camilla Banquet website. The header includes the company logo and navigation links. The profile card shows the user's name, email, and phone number, with a 'SAVE CHANGES' button at the bottom.

Personal Information	
	Fathima Safiya safiyasakab1@gmail.com UPLOAD NEW PHOTO
FIRST NAME	LAST NAME
Fathima	Safiya
EMAIL ADDRESS	VERIFIED
safiyasakab1@gmail.com	
PHONE NUMBER	
0705178558	
SAVE CHANGES	

Figure 3. 9 : Customer Profile Page

The Customer Profile Interface allows users to view and update their personal account information, including name, phone number, and billing details.

Updated information is saved to the user's Firestore profile document. These details are used by the system for customer communication, including booking confirmations, notifications, and receipts.

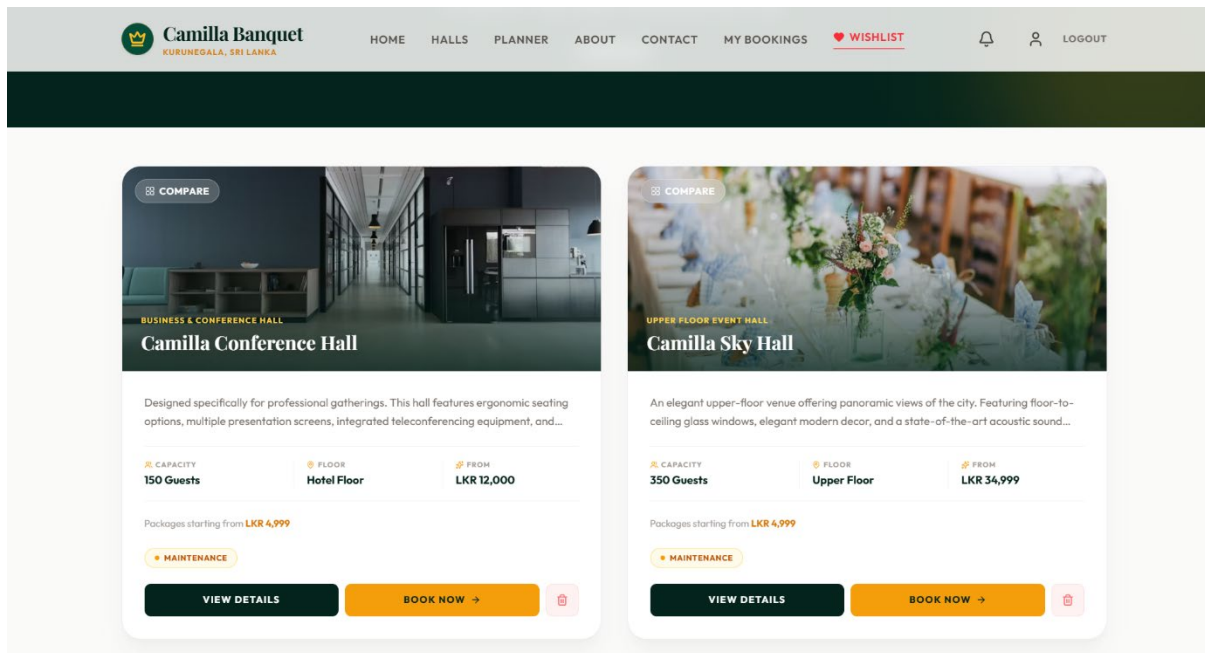


Figure 3. 10 : Wishlist Page

The Wishlist Interface provides customers with a personalized area to save preferred halls and packages for future consideration.

Saved items are stored within the customer's Firestore profile document and displayed in a similar layout to the Hall Catalog. Customers can review saved options, remove unwanted items, or continue to the booking process.

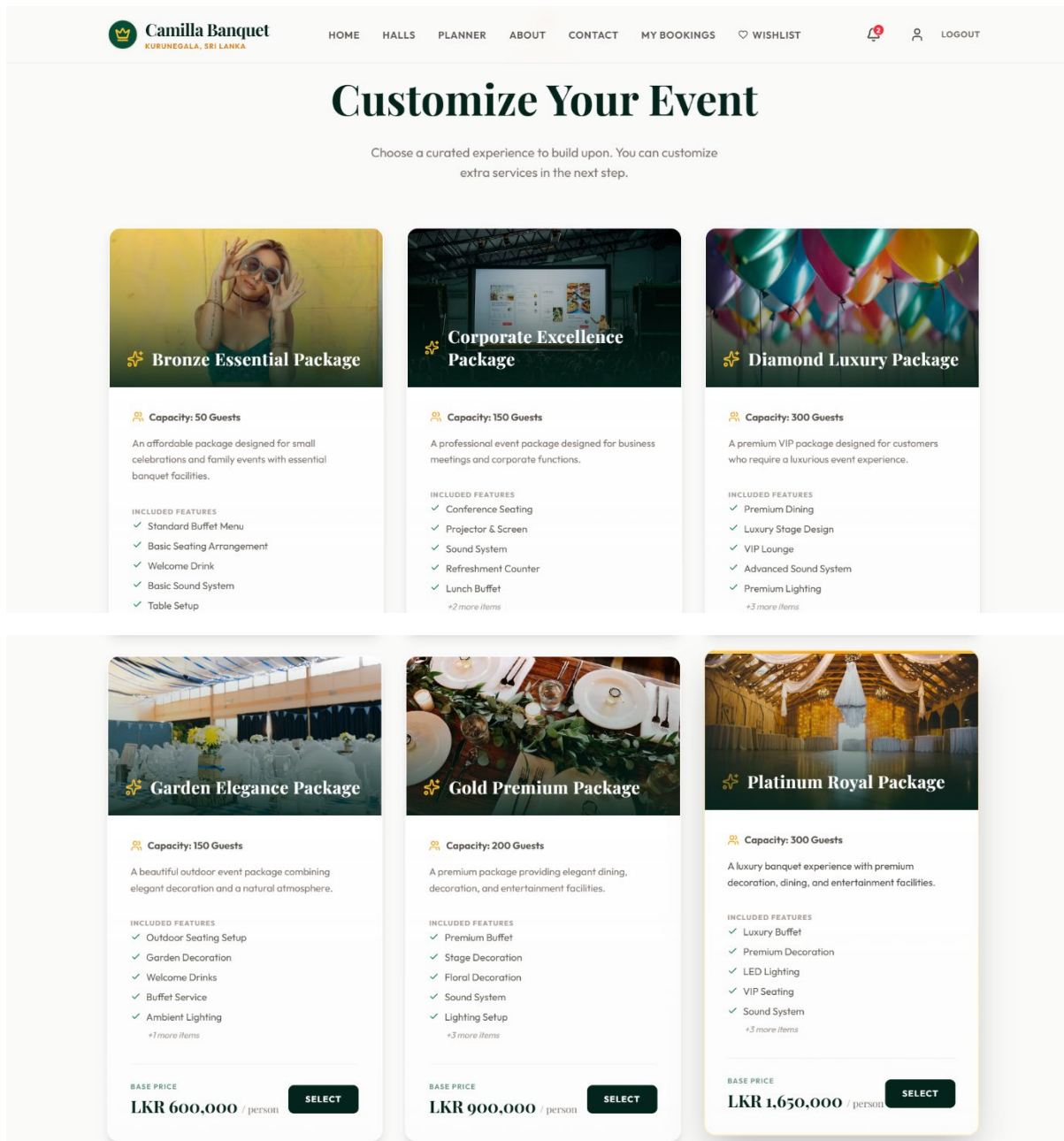


Figure 3. 11 : Event Customization Page

The Customize Your Event Interface allows customers to personalize their booking by selecting additional services before checkout. Users can choose optional services such as photography, DJ services, floral decorations, and catering through interactive service cards.

The interface dynamically updates the selected services and recalculates the estimated total cost based on customer choices. The final selected service details are passed to the checkout process, providing customers with better control over their event customization and budget.

Camilla Banquet
KURUNEGALA, SRI LANKA

HOME HALLS PLANNER ABOUT CONTACT MY BOOKINGS WISHLIST

✓ Hall ✓ Package **3 Review & Submit**

Review & Submit Booking

Please review your selections and confirm the event details.

Review Event Details

DATE: 8/30/2026

GUESTS: 65

EVENT TYPE: Corporate Meeting

[DOWNLOAD QUOTATION PDF](#)

[CONFIRM BOOKING REQUEST](#)

Booking Cost Summary

Full pricing breakdown

Hall Rental	LKR 12,000
Selected Package	LKR 227,500
GRAND TOTAL: LKR 239,500	
Advance Payment	LKR 47,900
Remaining Balance	LKR 191,600

Figure 3. 12 : Booking Review & Checkout Page

The Booking Review & Checkout Interface provides customers with a final summary of their reservation before confirmation. It displays important booking details including selected hall, event date, guest count, package details, additional services, and the total cost.

Customers can select their preferred payment method, such as online payment through PayHere or manual cash payment. The system validates the booking information before submission to prevent incomplete or incorrect reservations.

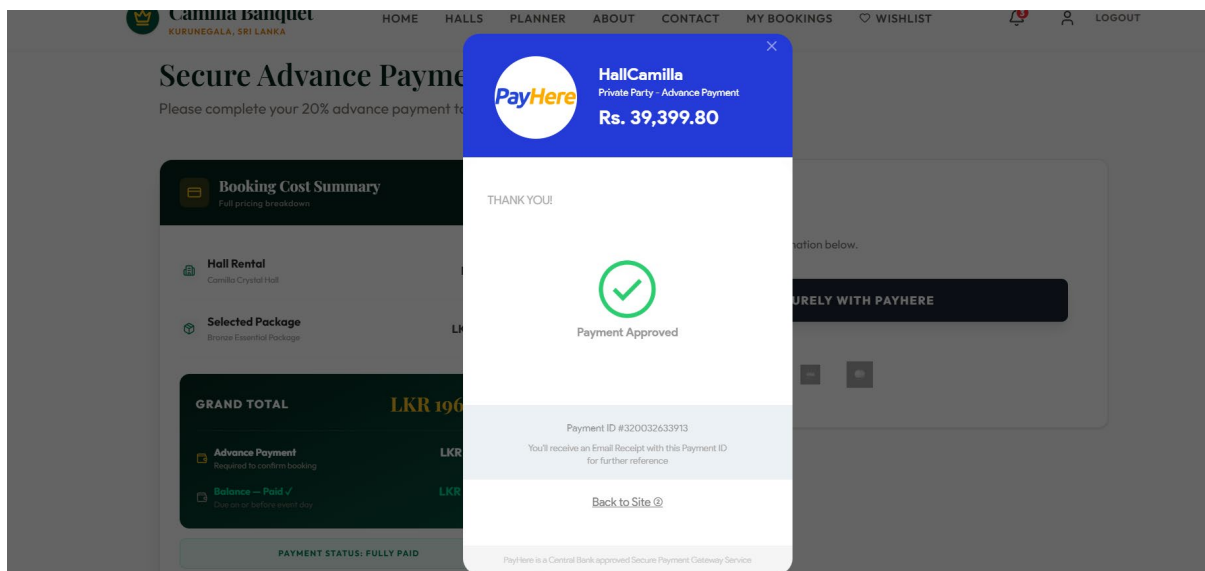
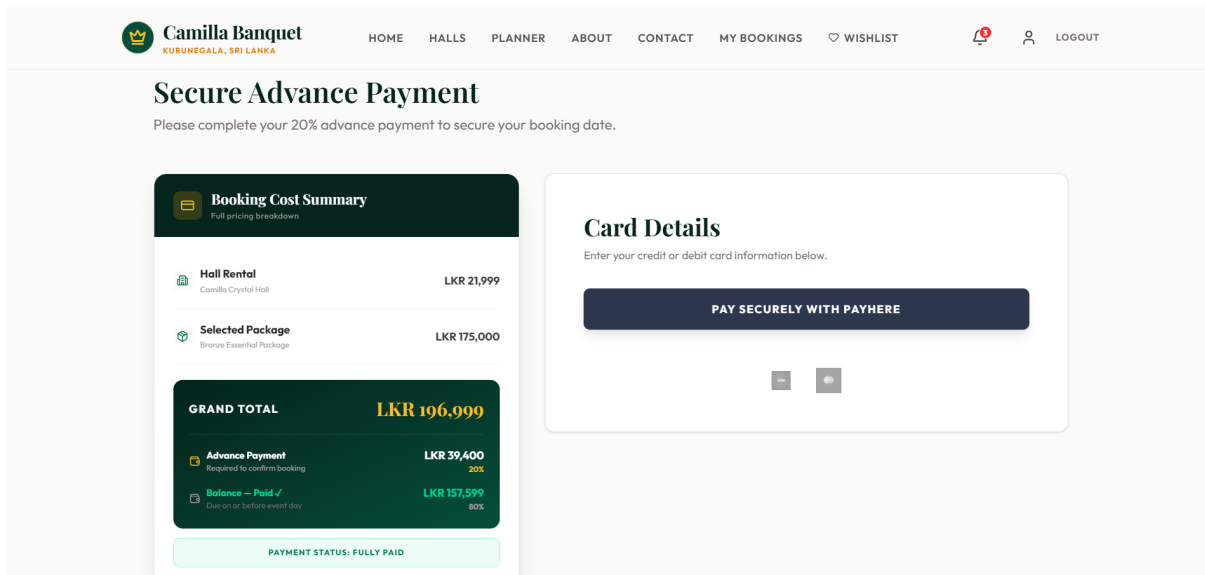


Figure 3. 13 : Payment Page

The Payment Interface enables customers to complete advance payments securely through the integrated PayHere payment gateway. It provides a simple payment screen where users can proceed with their selected payment option.

After successful payment verification, the system records the transaction details and creates the booking record in the Firebase Firestore bookings collection with the payment information.

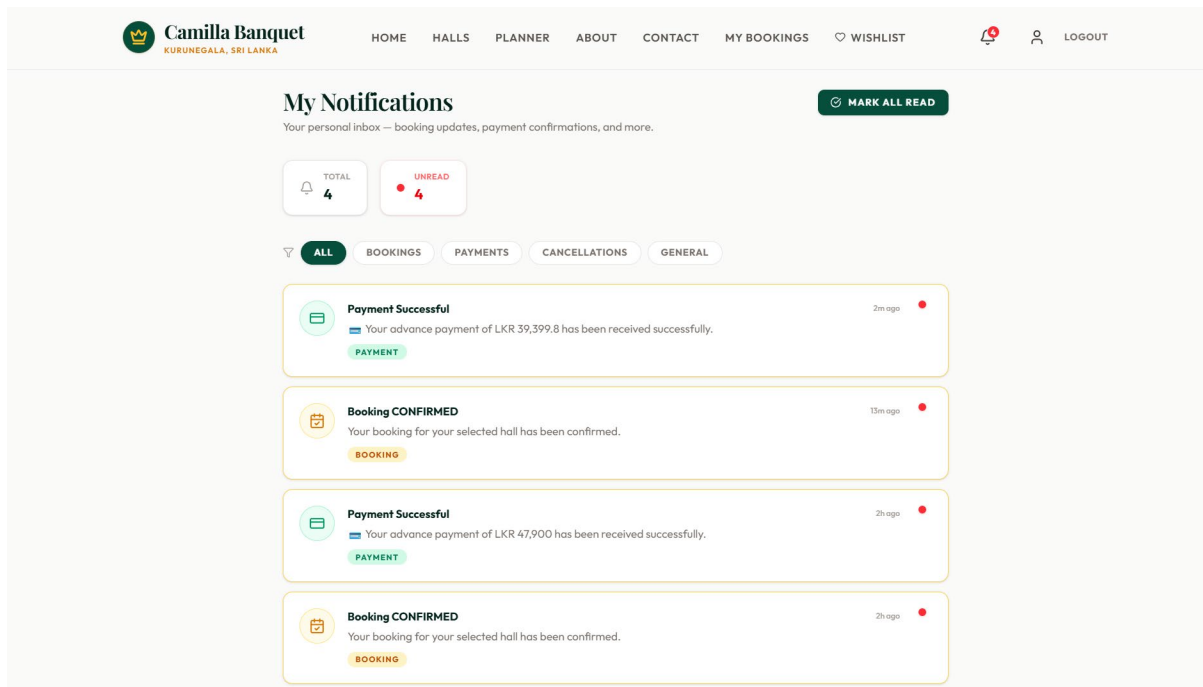


Figure 3. 14 : Customer Notifications Page

The Notifications Interface provides customers with a centralized location to view important system alerts and booking-related updates. It displays notifications in an organized list format, including details such as notification title, message preview, and timestamp.

The interface retrieves notification records from the Firebase Firestore notifications collection based on the logged-in user's ID. Customers can view updates such as booking approvals, payment confirmations, and reservation status changes directly within the system.

This feature ensures that customers do not miss important information, even if they do not access external email notifications, while improving communication between the system and users.

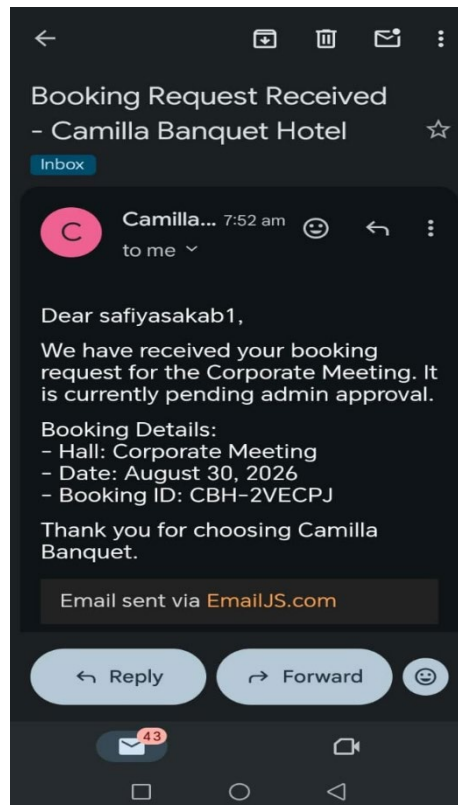


Figure 3. 15 : Automated Email Notification

The Automated Email Notification System uses EmailJS to send customers important updates such as booking confirmations, payment receipts, and booking status notifications. Each email includes relevant booking details and is sent to the customer's registered email address, providing a reliable record of their reservation and transactions.

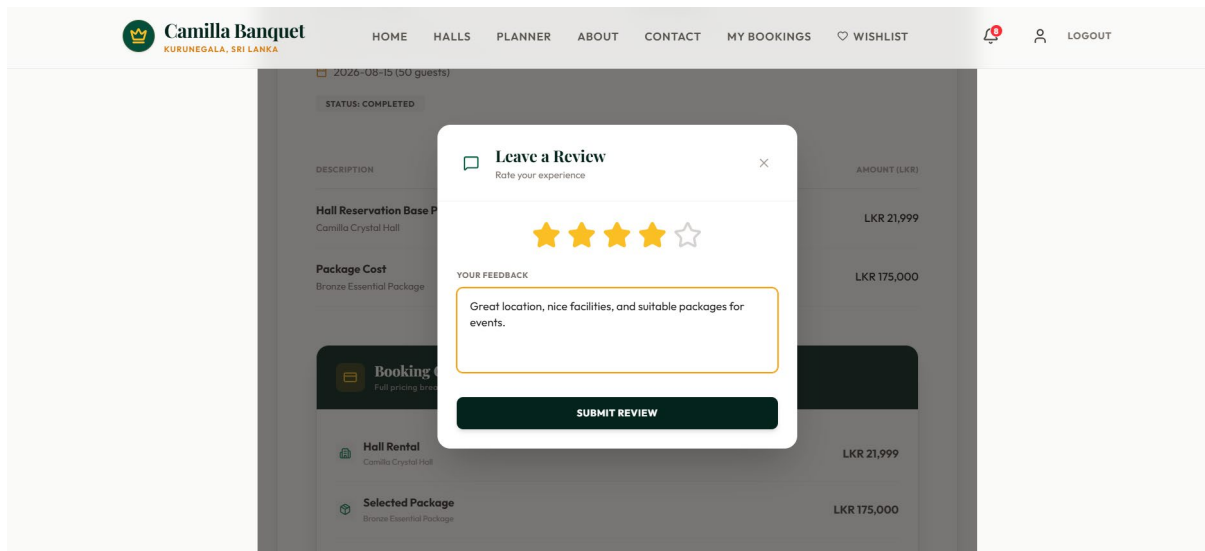


Figure 3. 16 : Customer Review Page

The Reviews Interface allows customers to share their experience after completing an event booking. Customers can provide feedback by selecting a star rating from 1 to 5 and entering comments about their experience.

The submitted reviews are stored in the Firebase Firestore reviews collection and linked with the customer's completed booking. New reviews are initially kept unpublished and require administrator approval before being displayed publicly, ensuring reliable and moderated feedback.

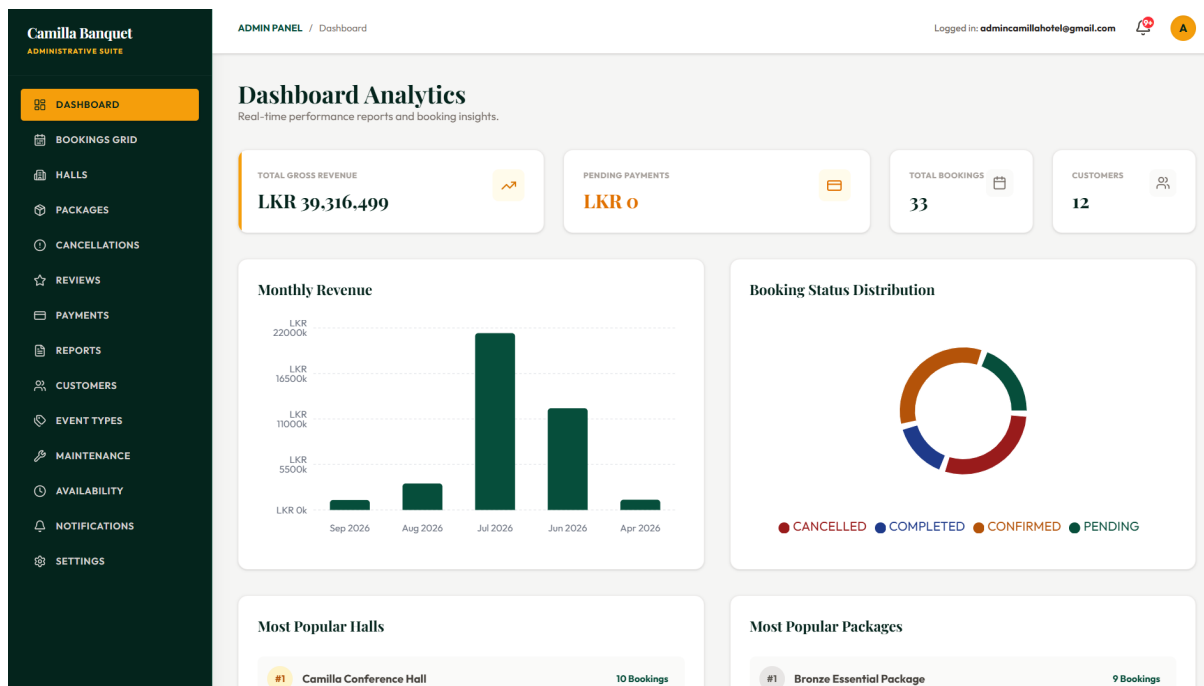


Figure 3. 17 : Admin Dashboard Page

The Admin Dashboard Interface acts as the central management area for administrators to monitor the overall performance of the hall booking system. It provides a visual overview of important business information through summary cards and analytical charts.

The dashboard displays key metrics such as total bookings, total revenue, and pending approvals. It uses graphical components to represent revenue trends and package popularity, helping administrators understand business performance easily.

The interface retrieves booking data from the Firebase Firestore bookings collection and processes information such as booking status and total prices to generate real-time reports. This helps administrators make informed decisions and efficiently manage daily operations.

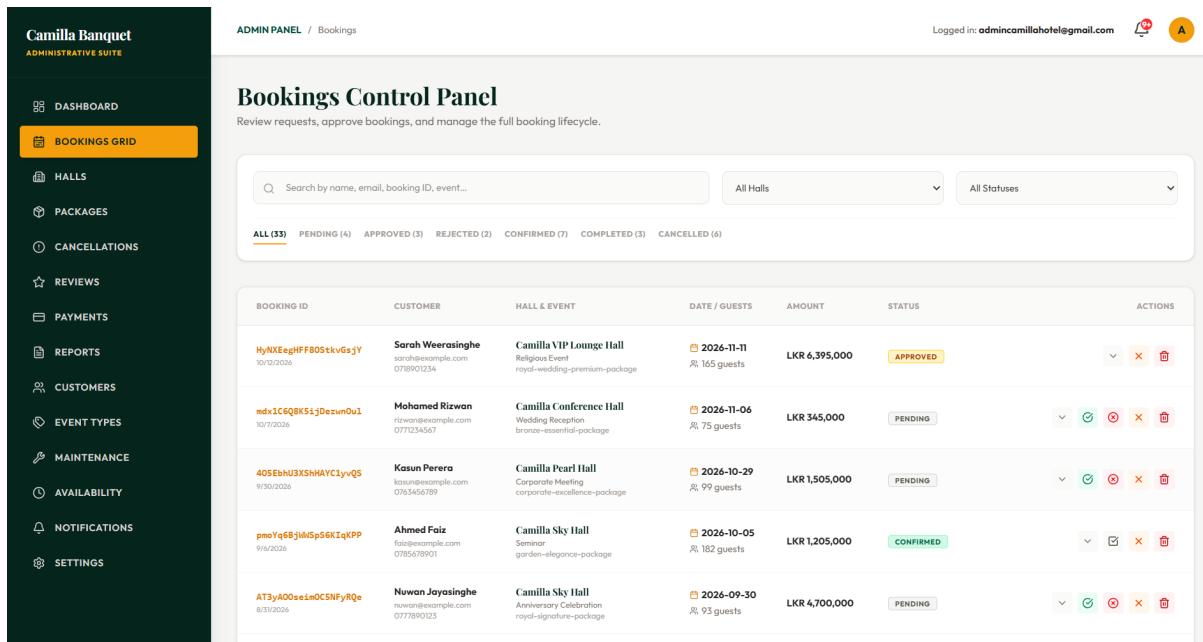


Figure 3. 18 : Booking Management Page

The Booking Management Interface allows administrators to efficiently manage customer reservations within the system. It provides a structured data table where administrators can view booking details, search records, and filter reservations based on their status, such as Pending, Approved, and Cancelled.

The interface retrieves booking information from the Firebase Firestore bookings collection and allows administrators to review and update reservation statuses through available actions. This replaces manual booking management methods with a centralized digital system, helping administrators process requests efficiently and avoid missed reservations.

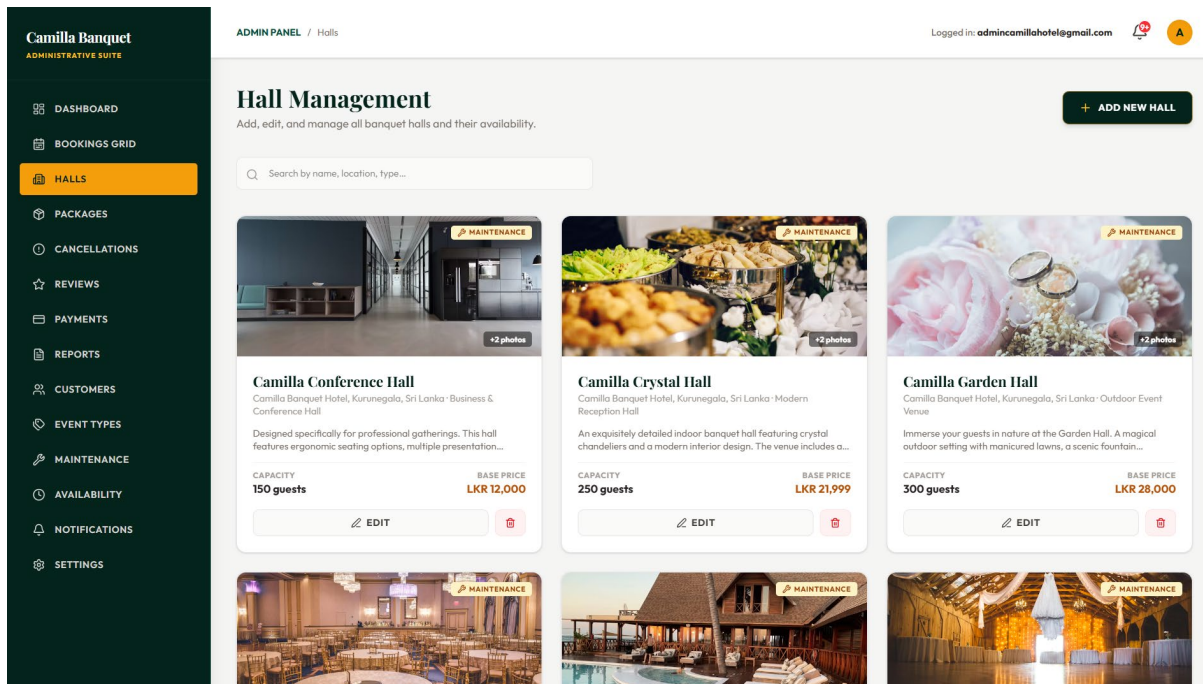


Figure 3. 19 : Hall Management Page

The Hall Details Management Interface allows administrators to manage and update detailed information about individual halls. It provides input fields for hall name, capacity, description, images, and other relevant details.

The interface includes form validation to ensure accurate data entry before saving changes. Updated information is stored in the Firebase Firestore halls collection and is automatically reflected in customer-facing interfaces such as the Hall Catalog and Hall Details pages.

This feature helps administrators maintain accurate hall information and keep the system content up to date.

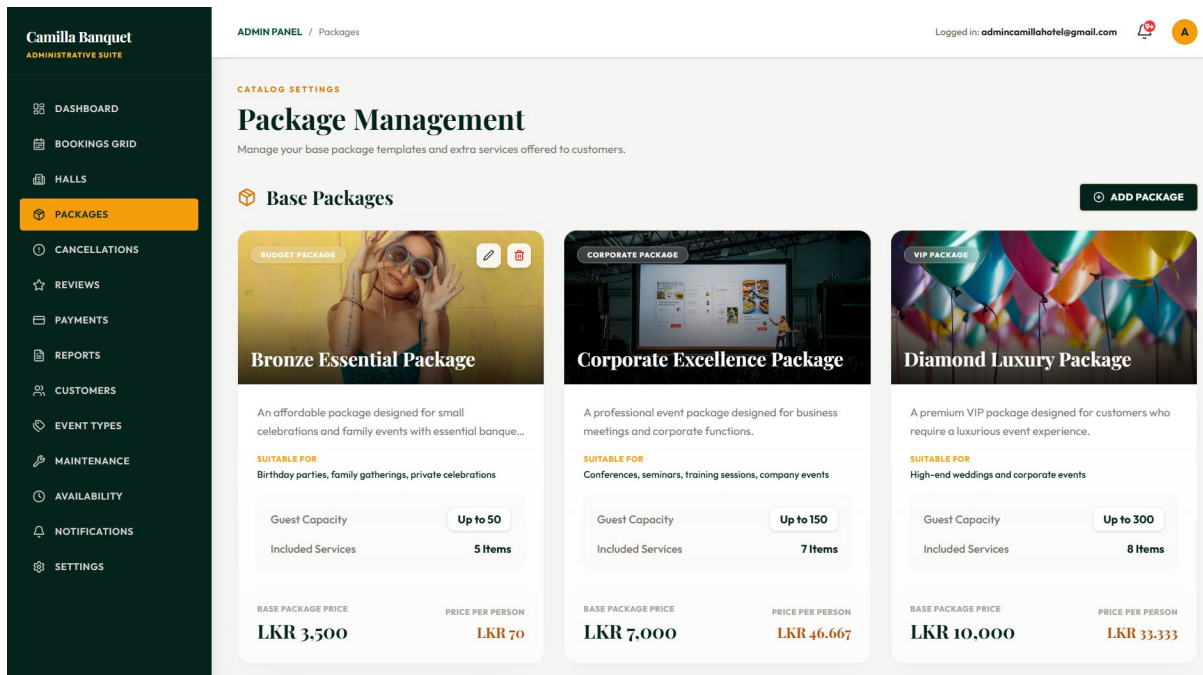


Figure 3. 20 : Package Management Page

The Package Management Interface allows administrators to create, update, and manage event packages offered by the venue. It provides a structured form for defining package details such as package name, category, guest limit, base price, extra guest charges, and included services.

The interface performs CRUD (Create, Read, Update, Delete) operations on the Firebase Firestore packages collection. Any changes made to package information, pricing, or included services are automatically reflected in the customer booking portal without requiring application updates.

This feature enables administrators to efficiently manage package offerings, adjust pricing, and introduce new event packages while ensuring customers always view the latest information.

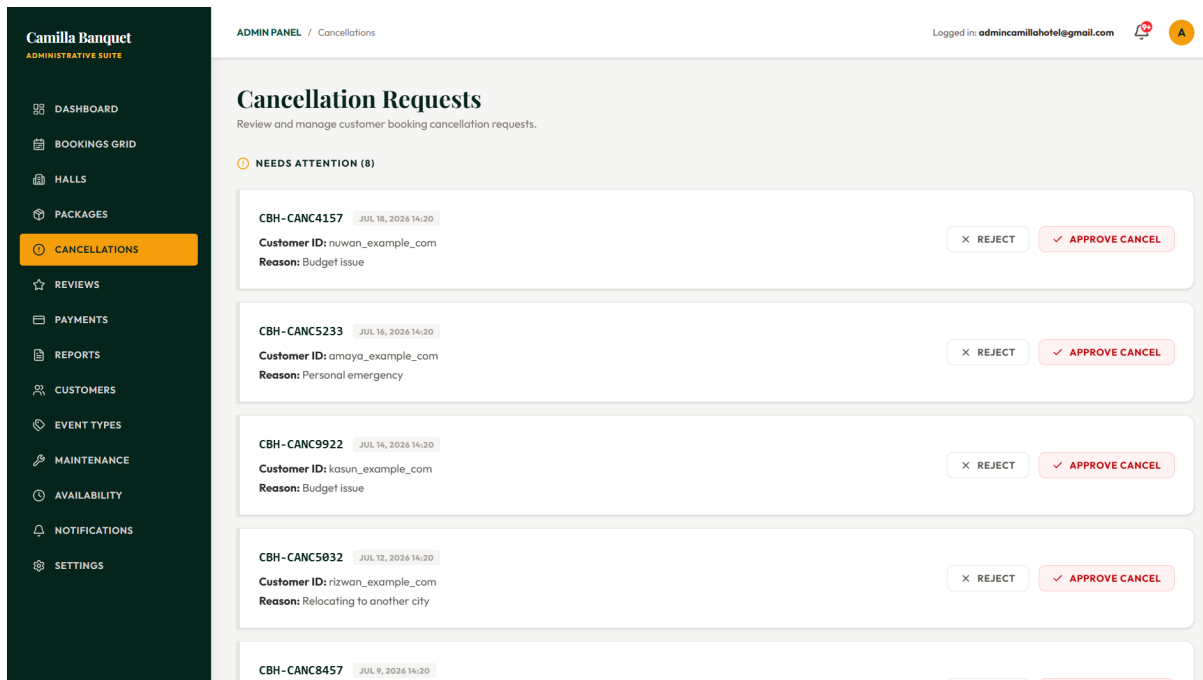


Figure 3. 21 : Booking Cancellation Page

The Cancellation Management Interface allows administrators to manage and cancel existing bookings when required. This functionality helps maintain accurate reservation records and ensures proper control over booking changes.

When an administrator selects the cancel option, the system displays a confirmation prompt to prevent accidental cancellations. After confirmation, the booking status is updated to "Cancelled" in the Firebase Firestore bookings collection.

Cancelled bookings are excluded from the availability checking process, allowing the selected date to become available for future reservations. This ensures that the booking calendar remains accurate and up to date.

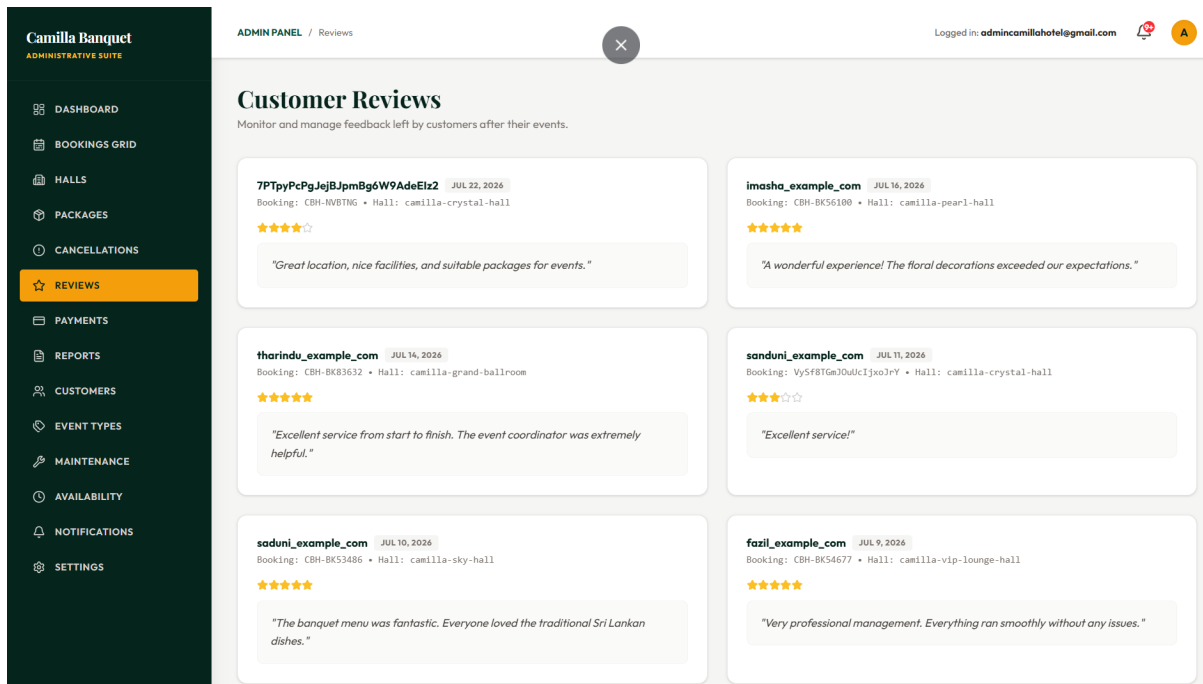


Figure 3. 22 : Review Management Page

The Reviews Management Interface allows administrators to review and control customer feedback before it appears publicly. It displays submitted reviews with details such as customer comments, star ratings, and publication status.

New reviews are stored with an unpublished status by default. Administrators can review the content and approve or reject reviews by changing the publication status. Only approved reviews are displayed on the customer-facing pages, ensuring reliable and appropriate feedback is shown publicly.

This feature helps administrators maintain the quality of customer reviews and protect the business reputation.

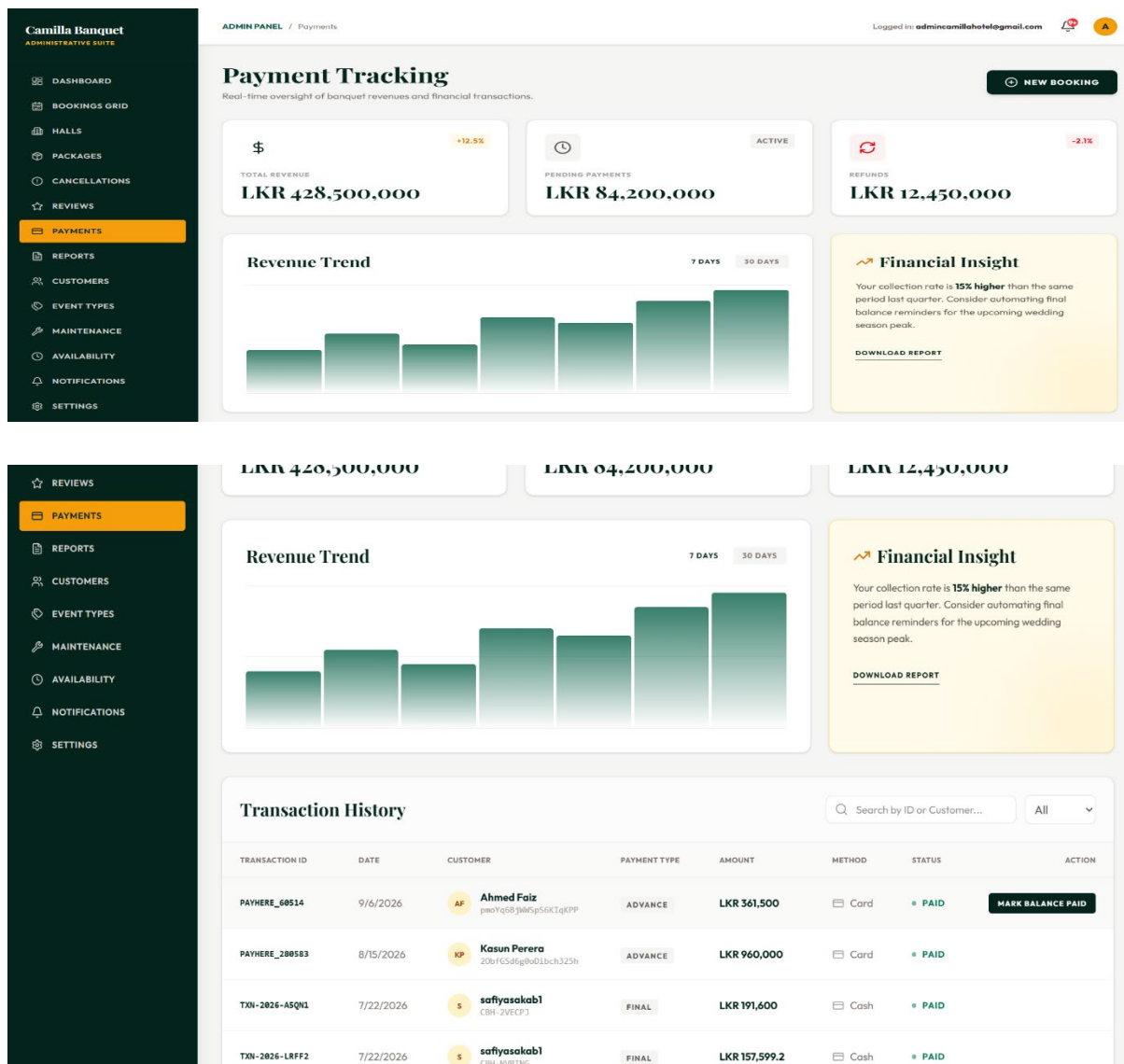


Figure 3. 23 : Payment Management Page

The Payment Management Interface allows administrators to monitor and manage customer payment records within the system. It separates bookings based on their payment status, helping administrators identify completed payments and pending transactions.

The interface highlights unpaid bookings, such as cash or bank transfer payments, and provides a "Mark as Paid" option to update the payment status after receiving payment. These updates are stored in the Firebase Firestore bookings collection, ensuring accurate financial records.

This feature helps administrators track outstanding payments efficiently and reduces dependency on manual payment records.

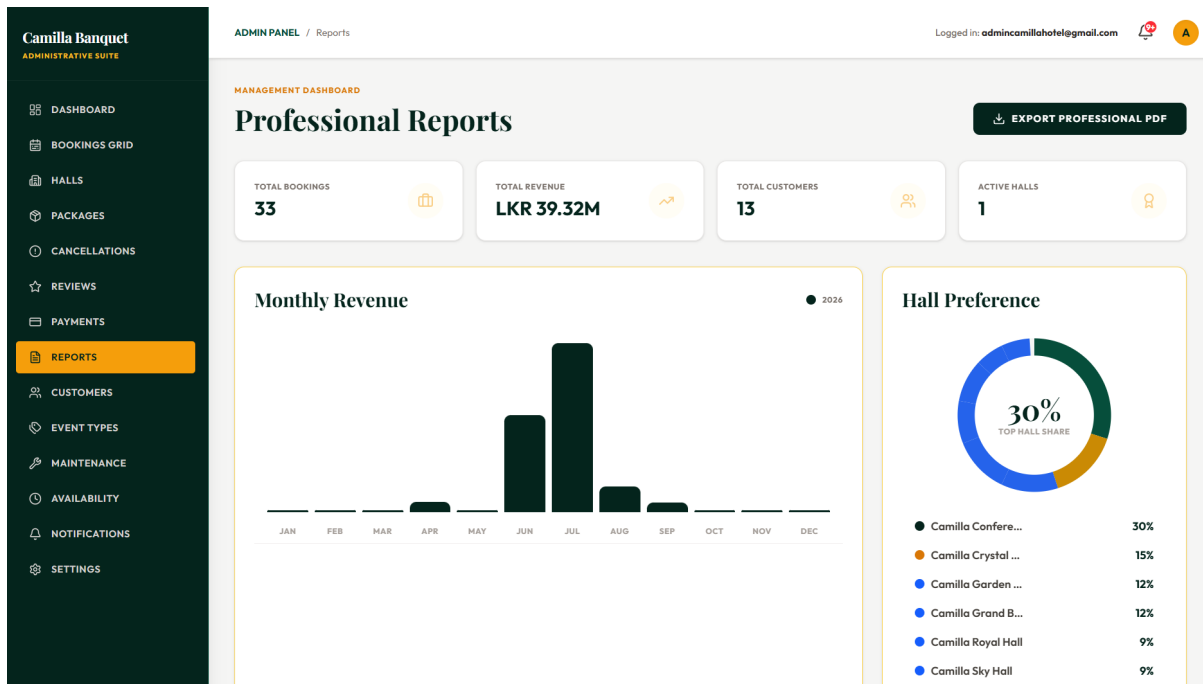


Figure 3. 24 : Reports & Analytics Page

The Reports & Analytics Interface allows administrators to generate professional reports for monitoring business performance and operational activities. It provides a date range selection option where administrators can specify the required reporting period.

When generating a report, the system retrieves relevant booking data from the Firebase Firestore bookings collection and processes the information into a structured format. Using jsPDF and jsPDF-AutoTable libraries, the system creates a downloadable PDF report containing important booking and financial details.

This feature reduces manual report preparation and helps administrators efficiently analyze booking activities, revenue information, and business performance.

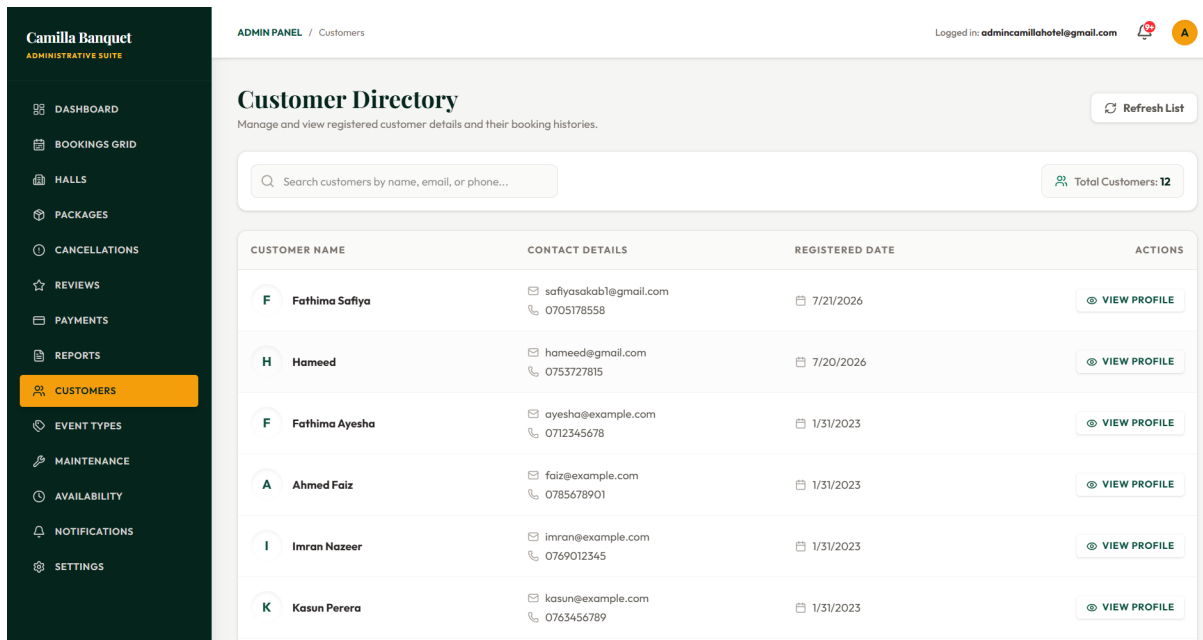


Figure 3. 25 : Customer Management Page

The Customer Management Interface provides administrators with a centralized view of all registered customers and their basic account information. It displays customer details such as name, email address, and phone number in an organized data table.

The interface retrieves customer records from the Firebase Firestore users collection and filters accounts based on the Customer role. A search feature allows administrators to quickly find specific customer profiles when required.

This feature helps administrators manage customer information efficiently and provides quick access to client details for handling booking-related assistance and support requests.

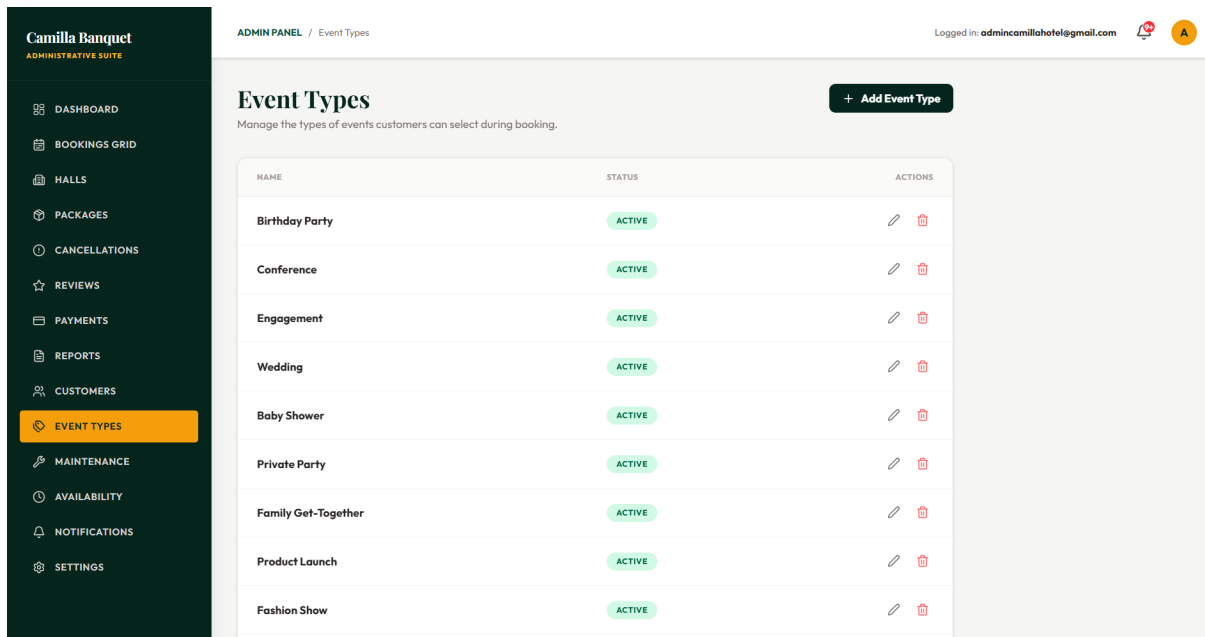


Figure 3. 26 : Event Types Management Page

The Event Types Management Interface allows administrators to manage available event categories such as weddings, corporate events, and birthday parties. Administrators can add, edit, activate, or deactivate event types through this interface.

The data is stored in the Firebase Firestore eventTypes collection, and changes are automatically reflected in the customer booking process. This feature provides flexibility by allowing event categories to be updated without modifying the application code.

Camilla Banquet

ADMINISTRATIVE SUITE

DASHBOARD

BOOKINGS GRID

HALLS

PACKAGES

CANCELLATIONS

REVIEWS

PAYMENTS

REPORTS

CUSTOMERS

EVENT TYPES

MAINTENANCE

AVAILABILITY

NOTIFICATIONS

SETTINGS

ADMIN PANEL / Maintenance

Logged in: admincamillahotel@gmail.com

A

Hall Maintenance

Schedule maintenance and block out dates for halls.

+ SCHEDULE MAINTENANCE

HALL	START DATE	END DATE	REASON	ACTIONS
Camilla Royal Hall	2026-09-28	2026-09-30	Deep cleaning	Remove
Camilla Sky Hall	2026-08-06	2026-08-10	Audio Equipment Tuning	Remove
Camilla Royal Hall	2026-09-08	2026-09-12	Pest Control Service	Remove
Camilla Pearl Hall	2026-08-21	2026-08-22	Wall Painting & Touchups	Remove
Camilla Grand Ballroom	2026-09-28	2026-09-29	Carpet Replacement	Remove
Camilla Garden Hall	2026-09-19	2026-09-23	HVAC Routine Check	Remove
Camilla Crystal Hall	2026-10-05	2026-10-07	Lighting System Upgrade	Remove
Camilla Conference Hall	2026-09-08	2026-09-09	Annual Deep Cleaning	Remove
Camilla Pearl Hall	2026-08-26	2026-08-28	Sound System Repair	Remove

Figure 3. 27 : Hall Maintenance Page

The Hall Maintenance Interface allows administrators to block unavailable dates due to repairs, private events, or temporary closures. Administrators can enter a maintenance reason along with the required start and end dates.

The maintenance records are stored in the Firebase Firestore maintenance collection and are considered during the booking availability check. This prevents customers from reserving halls during unavailable periods and helps maintain accurate scheduling.

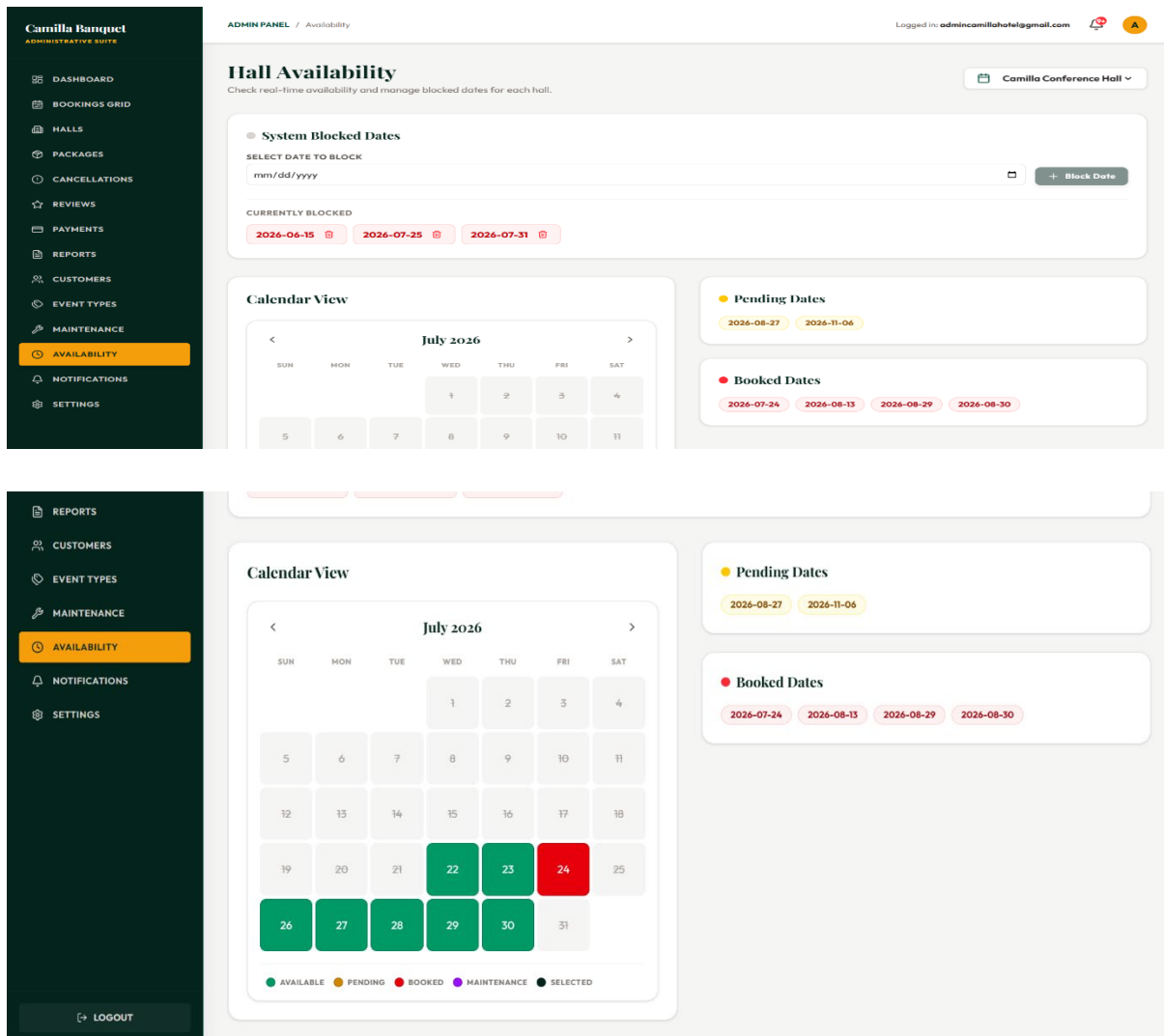


Figure 3. 28 : Availability Management Page

The Availability Management Interface provides administrators with a calendar-based view of hall schedules and bookings. It displays approved reservations as calendar events, allowing staff to quickly identify occupied and available dates.

The interface retrieves approved booking data from the Firebase Firestore bookings collection and presents it visually. This helps administrators manage venue capacity, verify availability, and handle customer inquiries efficiently.

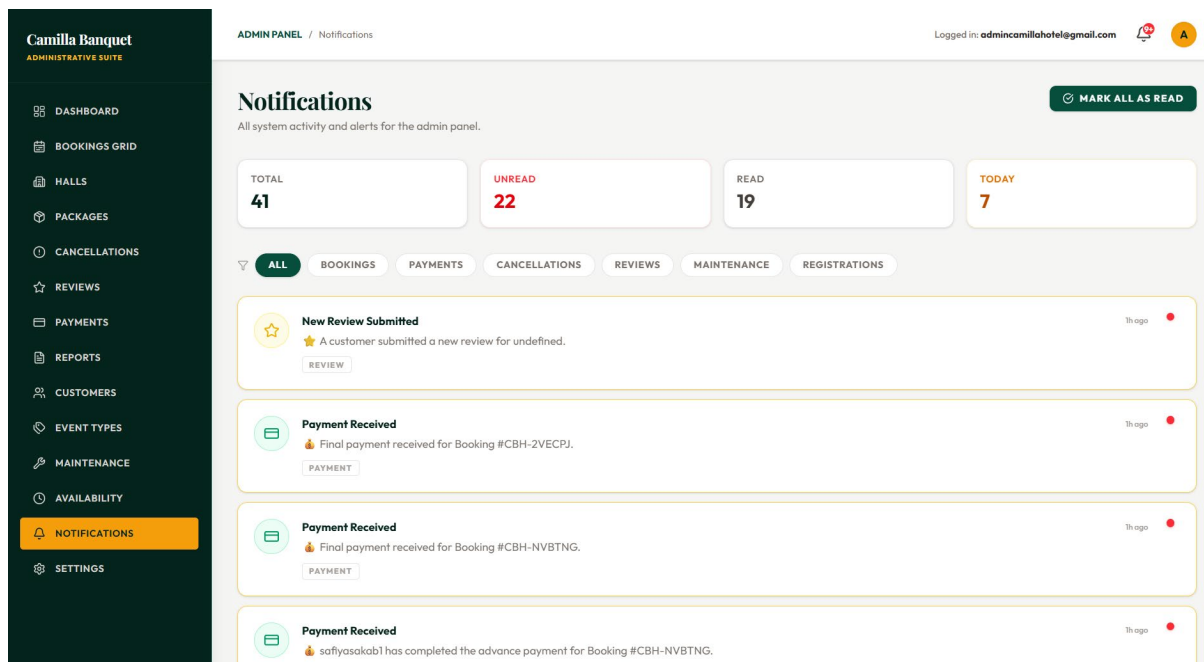


Figure 3. 29 : Notifications Management Page

The Notifications Management Interface allows administrators to monitor system-generated communications and maintain an activity record. It displays notification details such as recipient, message subject, and sent time.

The interface retrieves notification records from the Firebase Firestore notifications collection and provides a history of alerts and emails sent through the system. This helps administrators verify communication activities and handle customer inquiries effectively.

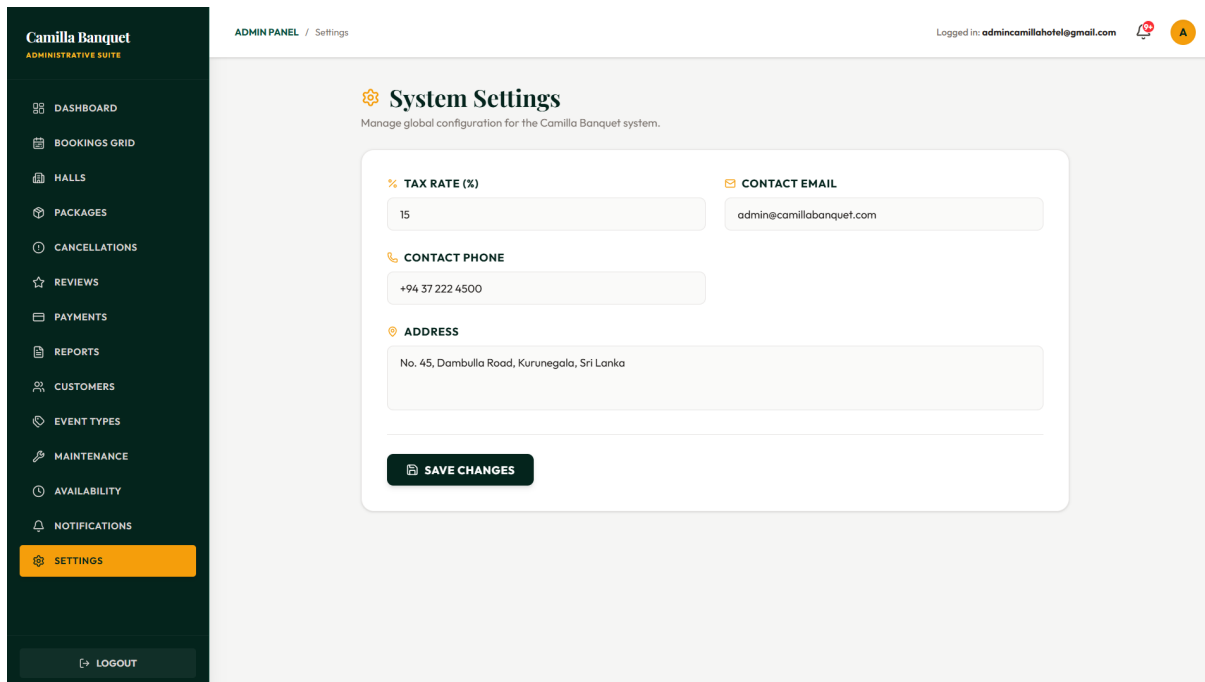


Figure 3. 30 : Admin Settings Page

The Settings Interface allows administrators to manage system preferences and account security details. It provides options to update administrative passwords and modify important venue information displayed across the system.

Password updates are securely handled through the Firebase Authentication service, ensuring that administrator accounts remain protected. This interface helps maintain system security and configuration management.

3.2 Database Design

HallCamilla utilizes Google Firebase Cloud Firestore, a scalable real-time NoSQL document database used for storing and synchronizing application data. Unlike traditional relational databases, Firestore stores information using Collections and Documents instead of tables and rows. However, the system maintains relational data consistency by using Document IDs and Reference IDs to establish logical relationships between different collections.

The database structure is designed to support hall management, customer reservations, payments, reviews, and administrative operations. The schema consists of the following main collections:

- **Users** – The central identity management collection that stores customer and administrator information. It contains fields such as uid (linked with Firebase Authentication), email, name, phone, role (Admin/Customer), and createdAt.
- **Halls** – Represents the available banquet halls within the system. It stores hall-related information including id, name, description, capacity, price, status, and imageUrls.
- **Packages** – Stores predefined event packages such as Silver, Gold, and Platinum. Each package contains details including id, hallId, packageName, packagePrice, guestLimit, extraGuestCharge, and includedServices.
- **Bookings** – The main transactional collection that records customer reservations. It stores booking information such as id, userId, hallId, packageId, eventDate, guestCount, totalAmount, bookingStatus (Pending, Approved, Confirmed, Completed, Cancelled), and paymentStatus.
- **Payments** – Maintains financial transaction records associated with bookings. It stores details including id, bookingId, userId, amount, paymentType (Advance/Final Balance), status, and date.
- **Services** – Stores optional additional services available for customers, such as photography, catering, and DJ services. It includes id, serviceName, description, price, pricingModel (Fixed/Per Person), and image.
- **Reviews** – Stores customer feedback and ratings related to completed events. It contains id, userId, hallId, bookingId, rating, comment, and createdAt.

- **CancellationRequests** – Manages customer booking cancellation requests and their approval workflow. It stores id, bookingId, userId, reason, status (Pending, Approved, Rejected), and requestDate.
- **Maintenances** – Stores hall maintenance schedules and blocked dates to prevent bookings during unavailable periods. It contains id, hallId, description, startDate, endDate, and status.
- **EventTypes** – Dynamically stores available event categories such as Wedding, Corporate Events, and Birthday Functions. It includes id, name, and isActive fields.

In this architecture, Users and Halls act as the primary parent entities. The Bookings collection functions as the central connection point, linking customers, halls, and packages together. Operational collections such as Payments, Reviews, and CancellationRequests store reference IDs such as userId, hallId, and bookingId to maintain relationships between documents.

Table 3.1 below summarizes the Document IDs (acting as Primary Keys), Reference IDs (acting as Foreign Keys), and relationship mapping between Firestore collections.

Table 3. 1 : Firestore Collection Structure and Relationship Mapping

Collection	Document ID (PK)	Reference ID (FK)	References	Relationship
Users	uid	—	—	Root/parent entity for customer and administrator identity
Halls	id	—	—	Root/parent entity for venue management
Packages	id	hallId	Halls.id	Many Packages → One Hall

Collection	Document ID (PK)	Reference ID (FK)	References	Relationship
Bookings	id	userId, hallId, packageId	Users.uid, Halls.id, Packages.id	Many Bookings → One User / One Hall / One Package
Payments	id	bookingId, userId	Bookings.id, Users.uid	Many Payments → One Booking
Reviews	id	userId, hallId, bookingId	Users.uid, Halls.id, Bookings.id	Many Reviews → One User / One Hall
CancellationRequests	id	bookingId, userId	Bookings.id, Users.uid	One Cancellation Request → One Booking
Maintenances	id	hallId	Halls.id	Many Maintenance Records → One Hall
Services	id	—	—	Independent entity used during booking customization
EventTypes	id	—	—	Independent entity used for event categorization

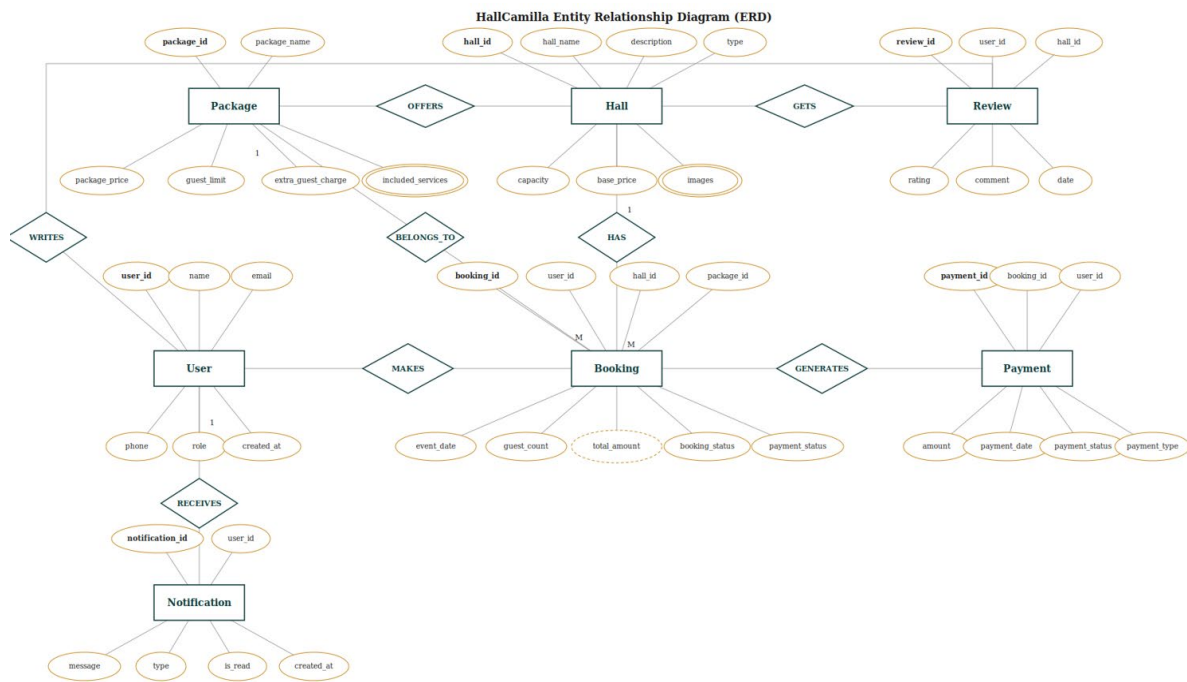


Figure 3. 31 : Entity Relationship Diagram

The Entity Relationship Diagram above illustrates how the core entities in the HallCamilla system are connected and interact with each other, even though Firebase Firestore does not enforce relationships through traditional foreign keys like a relational database management system. Instead, these relationships are maintained logically through reference fields stored within documents.

The Hall entity has a one-to-many relationship with the Booking entity through the HAS relationship. A single hall can be reserved through multiple booking requests over time, and this relationship is implemented by storing the corresponding hall_id field inside each booking document. Additionally, the Hall entity has a one-to-many relationship with the Package entity through the OFFERS relationship, as a single hall can provide multiple package tiers that customers can select during the booking process.

Similarly, the Package entity has a one-to-many relationship with the Booking entity through the BELONGS_TO relationship. A single service package, such as the Bronze Essential, Silver Celebration, or Garden Elegance package, can be selected by many different bookings, and this connection is maintained by storing the package_id field within each booking document.

The User entity has a one-to-many relationship with the Booking entity through the MAKES relationship, as a single registered customer can create multiple booking requests throughout

their usage of the system. Additionally, the User entity has a one-to-many relationship with the Notification entity through the RECEIVES relationship, because one user account can receive multiple notifications generated by different system activities.

The Booking entity has a one-to-many relationship with the Payment entity through the GENERATES relationship. A single confirmed booking can generate multiple payment records during its lifecycle, such as an advance payment and a final balance payment. The total_amount attribute in the Booking entity is represented with a dashed outline because it is a derived attribute. This means that the value is not directly entered by the user but is automatically calculated by the system based on the selected hall price and package price.

Understanding these relationships is essential for understanding how HallCamilla manages its business rules. When the system checks hall availability, calculates outstanding balances, retrieves customer notifications, or processes payment updates, it uses these reference fields to identify and retrieve the required data from the relevant Firestore collections.

The User collection stores profile information and access roles of all individuals registered within the platform. Each user document is identified by the user_id primary key and contains important details such as the user's name, email address, authentication credentials, contact number, and role information. The role field determines whether the account belongs to a customer or an administrator. During the login process, the system retrieves this information to determine the appropriate dashboard and access permissions for the user.

The Hall collection stores information about the physical banquet halls available for reservation at Camilla Banquet Hotel. Each hall document is uniquely identified by the hall_id primary key and contains details including the hall_name, description, maximum guest capacity, location, base price, hall type, and image information used for displaying the hall on customer-facing pages. Maintaining halls as a separate collection allows administrators to manage hall information, including pricing and availability details, from a centralized location through the Hall Management section of the administrator dashboard. This approach avoids unnecessary duplication of hall information across multiple booking records.

The Package collection stores the predefined event packages available for customers when making a reservation. Examples include the Bronze Essential, Silver Celebration, and Garden Elegance packages. Each package document is identified using the package_id primary key and stores information such as the package name, package price, description, and included

services. By maintaining packages as a separate collection, administrators can easily update package information through the Package Management section of the dashboard, ensuring consistency for all future bookings that select these packages.

The Booking collection represents the central operational component of the entire HallCamilla system because most major system activities depend on this collection. Each booking document contains the `booking_id` primary key, the `user_id` of the customer who created the booking, the selected `hall_id`, the selected `package_id`, the `event_date`, guest count, calculated `total_amount`, booking status, and payment status.

The `booking_status` field represents the current stage of a reservation and can contain values such as Pending, Approved, Rejected, Confirmed, Completed, or Cancelled. The `payment_status` field is maintained separately and can contain values such as Unpaid, Advance Paid, or Fully Paid. Separating these two fields is an important design decision because booking progress and payment progress are independent processes. For example, a booking may be approved by an administrator while the customer has not yet completed the required payment. Keeping these values separate allows the system to accurately represent real-world booking scenarios.

The Payment collection functions as the financial record system of HallCamilla by maintaining details of all payment transactions related to bookings. These payments may be processed through the PayHere sandbox payment gateway or recorded manually by administrators as cash payments. Each payment document contains the `payment_id` primary key, the related `booking_id`, payment amount, payment date, payment status, payment method, and payment type.

The `payment_method` field identifies whether the payment was completed through an online card transaction or a cash payment, while the `payment_type` field identifies whether the transaction represents an advance payment or a final payment. When a new payment document is created, the `paymentService.ts` service updates the corresponding booking document to ensure that the payment status remains synchronized between the Payment and Booking collections.

The Notification collection stores all system-generated alerts for both customers and administrators. Each notification document contains the `notification_id` primary key, the related `user_id`, message content, notification type, notification date, and notification status.

The `notification_type` field categorizes notifications into areas such as booking, payment, or general system updates, while the `status` field indicates whether the notification has been read.

Notifications are automatically created by backend services whenever important events occur, such as booking status changes or payment updates. Users only interact with this collection when marking notifications as read, ensuring that notification generation remains controlled by the system's business logic rather than being managed directly by the user interface.

The Review collection captures customer feedback regarding their experience after using the hall booking service. Each review document contains the `review_id` primary key, the related `user_id`, the `hall_id`, a numerical rating, a written comment, and the date on which the review was submitted. The User entity has a one-to-many relationship with the Review entity through the WRITES relationship, as a customer can submit multiple reviews over time for different booking experiences. Similarly, the Hall entity has a one-to-many relationship with the Review entity through the GETS relationship, allowing the system to collect, manage, and display customer feedback associated with each specific hall.

3.3 Architectural Design

HallCamilla follows a modern, serverless client architecture, sometimes described as a decoupled frontend combined with a Backend-as-a-Service (BaaS) model, rather than the traditional pattern of a custom-built backend server communicating with a separate database. This architectural choice was made deliberately, since it allowed the project's development effort to be concentrated on the business logic and user experience of the booking and payment workflow, instead of being spent on building, securing, and hosting a custom API server from scratch.

HallCamilla System Architecture

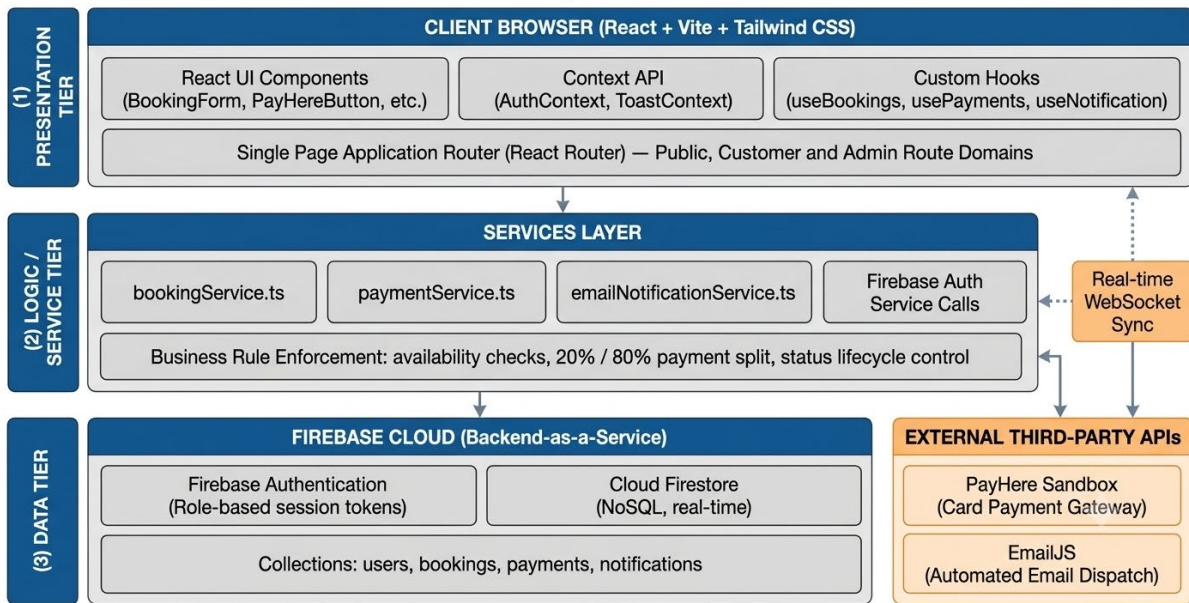


Figure 3. 32 : System Architecture Diagram

The architecture diagram above divides the system into three logical tiers. The Presentation Tier represents everything that runs inside the customer's or administrator's web browser, including the React user interface components such as the booking form and the PayHere payment button, the global Context API providers that hold shared state such as the currently authenticated user, and the custom hooks that expose live database subscriptions to the rest of the interface.

Beneath this sits the Logic and Service Tier, which contains dedicated service files such as `bookingService.ts`, `paymentService.ts`, and `emailNotificationService.ts`. This tier is deliberately placed between the user interface and the backend so that important business rules, including availability checking, the twenty and eighty percent payment split, booking status lifecycle management, and notification generation, are handled consistently in one central location rather than being scattered across individual React components.

Finally, the Data Tier represents the Firebase cloud infrastructure itself, consisting of Firebase Authentication, which verifies user identity and access roles, and Cloud Firestore, which stores and retrieves all application data. The diagram also shows how the Service Tier communicates with two external third-party services: EmailJS for sending automated email notifications and PayHere Sandbox for processing online card payments. It also illustrates the real-time WebSocket connection maintained by Firestore with the Presentation Tier, which allows

changes such as administrator booking approval actions to appear on the customer's dashboard immediately without requiring a manual browser refresh.

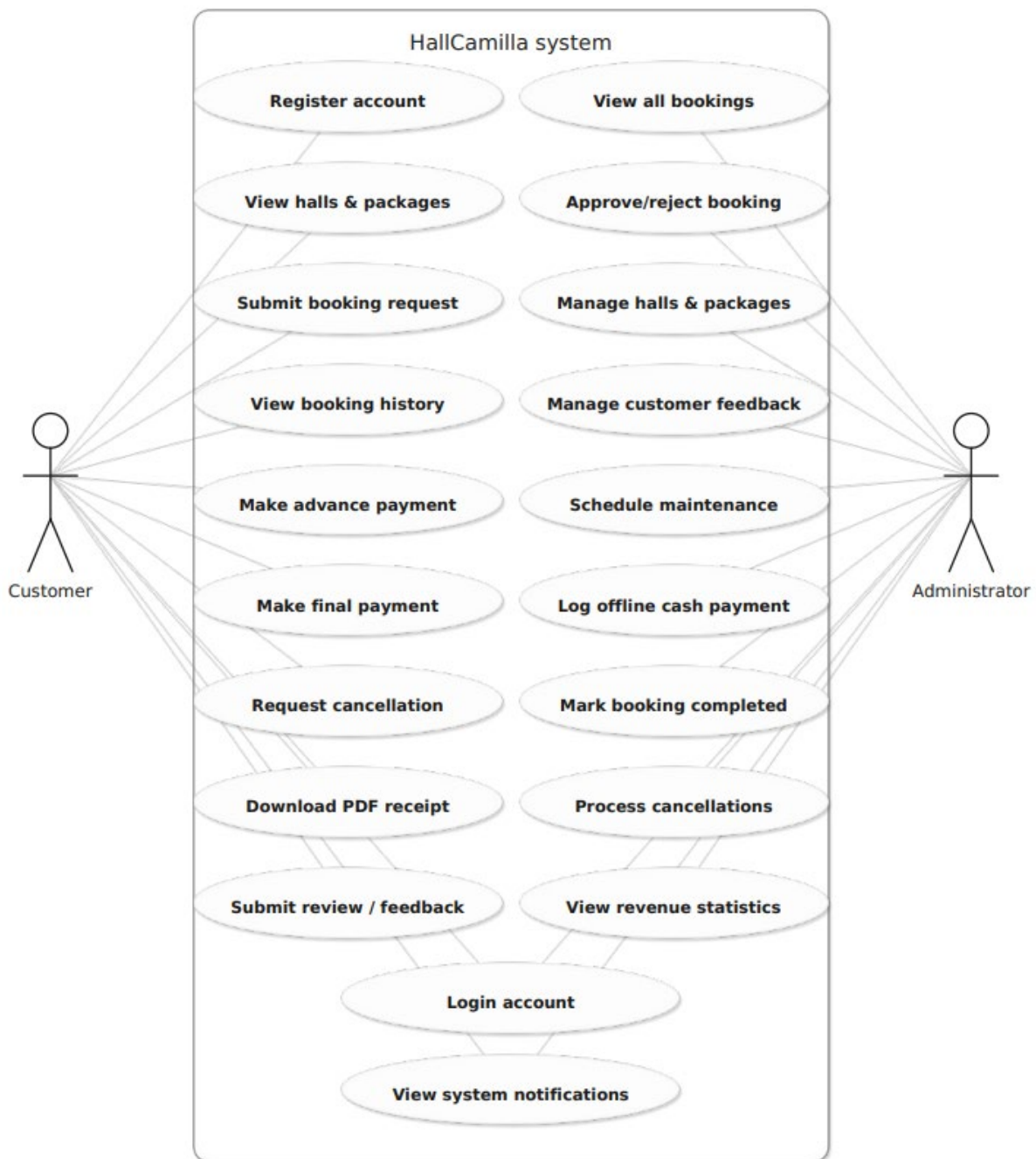


Figure 3. 33 : Use Case Diagram

The use case diagram illustrates the two main actors who interact with HallCamilla and the specific operations each actor can perform within the system boundary. There is no unauthenticated guest actor shown in the diagram because HallCamilla requires users to

authenticate before accessing dashboards or submitting booking requests. This ensures that all meaningful actions within the system are connected to a specific user account.

The Customer actor is shown on the left side of the diagram and is connected to use cases such as registering an account, submitting a booking request, making advance or final payments, requesting booking cancellations, and downloading PDF receipts of completed transactions. The Administrator actor is shown on the right side and is connected to use cases such as approving or rejecting booking requests, recording offline cash payments, marking bookings as completed, and viewing revenue statistics through the payment dashboard.

Several use cases, including user authentication and notification viewing, are shared between both actors because customers and administrators must log into the system and both receive their own category of real-time notifications.

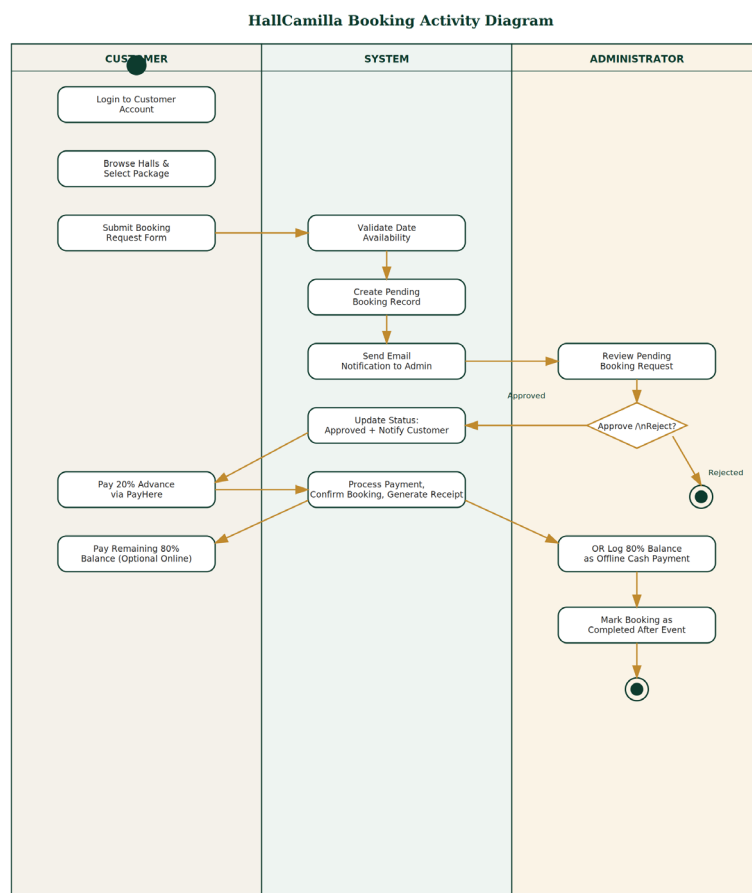


Figure 3. 34 : Activity Diagram

The activity diagram traces the complete lifecycle of a single booking from the moment a customer logs into the system until the event is completed. The process is organised into three swimlanes representing the Customer, System, and Administrator respectively.

The flow begins in the Customer lane, where the user logs in, browses available halls and packages, and submits a booking request. Control then moves into the System lane, where the request is validated against existing reservations for the selected date, a new document is created in the bookings collection with a Pending status, and an email notification is automatically sent to the hotel administrator.

The process then moves into the Administrator lane, where the booking request is reviewed and a decision is made at the Approve or Reject decision point. A rejected request ends the process at that stage, while an approved request causes the System lane to update the booking status and notify the customer. Control then returns to the Customer lane, where the user completes the twenty percent advance payment through PayHere. The System processes the transaction, updates the payment information, confirms the booking, and generates a payment receipt.

Later in the timeline, the remaining eighty percent balance payment can either be completed online by the customer or recorded as an offline cash payment by the administrator. Finally, the administrator marks the booking as Completed once the event has taken place, after which the customer can log back into their dashboard to submit a post-event review rating their experience.

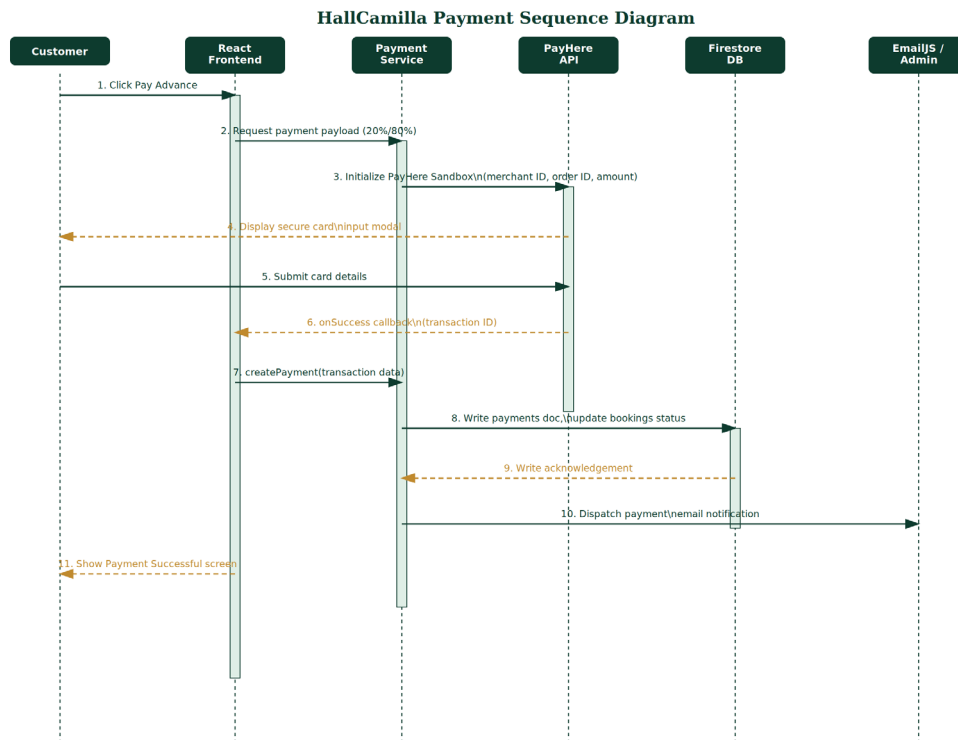


Figure 3. 35 : Sequence Diagram

The sequence diagram focuses specifically on the hybrid online payment process, as this is the most technically involved interaction in the system and the process that depends most heavily on communication with an external third-party service.

Reading the diagram from top to bottom, the process begins when the Customer clicks the Pay Online button inside the React Frontend. The frontend then communicates with the Payment Service to calculate the correct payment amount for either the twenty percent advance payment or the eighty percent final balance payment, depending on the current stage of the booking.

The Payment Service initializes the PayHere Sandbox payment widget using required details such as the merchant identifier, order identifier, and calculated payment amount. At this point, PayHere takes control and displays its secure payment interface directly to the customer. After the customer completes the card payment, PayHere returns an asynchronous success callback to the React Frontend containing transaction information. This response is represented using a dashed arrow in the sequence diagram because it represents a returned result rather than a new request.

The frontend then calls the Payment Service's createPayment function, which creates a new document in the payments collection within Firestore and updates the related booking

document with the latest payment status. After Firestore confirms the successful database update, the Payment Service triggers an automated email notification through EmailJS to inform the hotel administrator about the received payment.

At the same time, the React Frontend displays the payment success confirmation message to the customer, completing the advance payment workflow.

CHAPTER 4 : IMPLEMENTATION

4.1 Development Approach

The development of the HallCamilla system followed an Agile Incremental development approach, where the system was developed through short iterative cycles. Each iteration focused on implementing and testing a specific functional module before proceeding to the next stage.

The development process began with the implementation of Firebase Authentication and Firestore database design, as these components provided the foundation required for user management, role handling, and data storage. After establishing the core structure, major functionalities such as hall management, booking processing, payment integration, and notification services were implemented incrementally.

Version control was managed using Git and GitHub, allowing changes to be tracked throughout the development process. The frontend application was continuously deployed using Firebase Hosting, enabling frequent testing of newly implemented features in a live environment.

4.2 Implementation of Key Modules

Authentication and Role Management

Authentication was implemented using Firebase Authentication with email and password-based login. Since Firebase Authentication does not provide built-in role management, a separate user document was created in the Firestore users collection during registration.

The user document stores additional information such as user details and role information (Customer/Admin). This role-based structure allows the React Router protection system to control access to different areas of the application, ensuring customers and administrators are directed to their respective interfaces.

Booking Management

The Booking Management module was implemented using a multi-step booking workflow that allows customers to select a hall, package, guest count, event date, and additional services.

Before creating a booking record, the system performs availability validation by checking existing bookings stored in the Firestore bookings collection. This prevents multiple customers from reserving the same hall for the same date.

The final booking amount is calculated dynamically based on selected package pricing, guest count, and additional service charges. The calculated value is stored with the booking record for future reference.

Payment Processing

The payment module was integrated using the PayHere Sandbox payment gateway to support online advance payments. The system calculates the required payment amount based on the booking payment rules and sends the payment details securely to the PayHere payment interface.

After successful payment confirmation, the system records the transaction details in the payments collection and updates the related booking payment status. Manual cash payments processed by administrators follow the same database structure to maintain consistent financial records.

Automated Email Notifications

Automated notifications were implemented using EmailJS to provide booking and payment-related communication between the system and users.

When important actions occur, such as booking submission, approval, or successful payment completion, the system triggers an email notification to the relevant users. This approach provides automated communication without requiring a dedicated email server.

Real-Time Dashboard Updates

The system utilizes Firestore real-time listeners through the `onSnapshot()` method to provide live data updates. When administrators modify booking statuses, changes are automatically reflected in the customer interface without requiring manual page refreshes.

This real-time functionality improves user experience by ensuring that customers and administrators always view the latest booking information.

4.3 Challenges Faced

Preventing Double Bookings

Since Firestore does not provide SQL-style unique constraints, preventing duplicate hall reservations required implementing a custom availability validation mechanism.

The system checks existing bookings before creating a new reservation and verifies whether the selected hall is already reserved for the requested date.

Managing Payment Status Flow

Handling different payment stages was challenging because the system needed to distinguish between booking approval, advance payment, and final payment completion.

To overcome this issue, separate fields were maintained for bookingStatus and paymentStatus. This ensured that payment updates did not incorrectly change the overall booking lifecycle.

Securing Administrative Routes

Since the application uses a client-side architecture, unauthorized users could attempt to access administrative routes directly.

A protected routing mechanism was implemented using a custom ProtectedRoute component, which verifies the authenticated user's role before granting access to administrative pages.

Managing Development Time and Feature Prioritization

Developing a complete booking management system within the available time required careful prioritization of features.

The development sequence was planned by implementing essential components first, including authentication, database structure, booking workflow, and user interfaces. Advanced integrations such as payment processing and automated notifications were implemented after the core system functionality was completed.

CHAPTER 5 : TESTING

5.1 Testing Approach

Testing was performed throughout the development process rather than only at the final stage. Each implemented module was tested against its expected functionality to ensure reliability and correctness. Testing was conducted using the deployed **Firebase Hosting environment** to validate system behavior under realistic operating conditions.

The following three levels of testing were carried out:

Unit Testing

Individual system functions were tested independently, including booking cost calculations, advance payment calculations, and hall availability validation logic. This ensured that core business rules produced accurate results.

Integration Testing

Integration testing was performed to verify communication between different system components, including Firebase Authentication, Firestore database operations, PayHere Sandbox payment processing, and EmailJS notification services. Both successful and failed scenarios were tested to ensure proper error handling.

User Acceptance Testing (UAT)

Complete user workflows were tested from the customer's and administrator's perspectives. The testing process included user registration, login, browsing halls, package selection, booking submission, administrator approval, online payment processing, and submitting post-event reviews.

5.2 Test Cases

Table 5.1 presents a representative set of test cases covering the major functional requirements of the HallCamilla system.

Table 5 . 1 : Representative Test Cases

Test ID	Module	Test Description	Expected Result	Status
TC-01	Authentication	Register with a valid email and password	Firebase authentication account is created, user profile is stored in Firestore, and user is redirected successfully	Pass
TC-02	Authentication	Login using an incorrect password	System rejects access and displays an appropriate error message	Pass
TC-03	Booking	Attempt to book an already reserved hall date	System prevents booking and displays an availability warning	Pass
TC-04	Booking	Submit a booking request for an available date	Booking is stored with Pending status and notification is generated	Pass
TC-05	Admin Panel	Administrator approves a pending booking	Booking status is updated and reflected immediately in the customer dashboard	Pass
TC-06	Payment	Complete advance payment through PayHere Sandbox	Payment record is stored and booking payment status is updated successfully	Pass
TC-07	Payment	Administrator records final cash payment	Payment record is updated and booking is marked as fully paid	Pass

Test ID	Module	Test Description	Expected Result	Status
TC-08	Notifications	Change booking status from Pending to Approved	Customer receives an in-app notification about the status update	Pass
TC-09	Security	Customer attempts to access admin URL	Protected route prevents access and redirects the user appropriately	Pass

5.3 User Acceptance Testing Summary

The complete customer and administrator workflows were successfully tested using the live Firebase environment. The main booking process, including registration, hall selection, booking submission, administrator approval, payment processing, and review submission, was verified successfully.

During testing, several minor issues were identified, including delays in real-time status updates and routing behavior for unauthorized users. These issues were analyzed, corrected, and retested before project completion.

After final testing, no critical errors or blocking issues remained. The completed HallCamilla system demonstrated stable functionality and satisfied the defined functional requirements.

CHAPTER 6 : RESULTS AND EVALUATION

6.1 Achievement of Objectives

- The objectives defined at the beginning of the project were successfully achieved, and the implemented features are fully functional within the deployed HallCamilla system.
- The project successfully developed a complete serverless web-based hall booking and management system using React, Vite, Tailwind CSS, Firebase Authentication, and Firestore, which has been deployed using Firebase Hosting.
- Customers are able to browse available halls, view package details and pricing information, calculate estimated event costs, and submit booking requests with real-time availability validation to prevent duplicate reservations.
- Administrators can manage the booking process through a real-time dashboard powered by Firestore listeners, allowing them to monitor, approve, or reject customer reservations without requiring manual page refreshes.
- The system successfully integrates the PayHere Sandbox payment gateway to support online advance payments while maintaining the required payment workflow of a 20% advance payment and 80% remaining balance.
- Automated email notifications are implemented using EmailJS, allowing customers and administrators to receive updates regarding booking confirmations, approvals, and payment-related activities.
- The system also provides a responsive user interface, enabling customers and administrators to access and manage the platform effectively across desktop and mobile devices.

6.2 Limitations

- Although the HallCamilla system achieved its main objectives, several limitations were identified due to the project scope and available development time.
- The PayHere payment integration currently operates in Sandbox mode. Although it successfully demonstrates the payment workflow, real-world transactions would require official merchant verification and production configuration.

- Email notifications are implemented using EmailJS, which provides a simple client-side communication solution. However, large-scale commercial usage may require migration to a dedicated email service such as SendGrid or Amazon SES to support higher email volumes.
- The current system does not include SMS notification functionality. Users receive important updates through the application notification system and email communication only.
- The payment workflow follows a fixed 20% advance and 80% final balance structure. Flexible installment plans, automated refunds, and advanced cancellation refund calculations are not currently supported and would require additional development.
- Testing was mainly performed manually during the development process. Although this was sufficient for an academic project, a commercial-level deployment would benefit from automated testing frameworks such as Jest or Cypress and a dedicated quality assurance process.
- The evaluation was mainly conducted by the developer and project supervisor. A larger user evaluation involving external customers and hotel staff would provide additional insights into usability and further improvements.

CHAPTER 7 : CONCLUSION AND FUTURE WORK

7.1 Conclusion

The HallCamilla system was developed to address the challenges associated with traditional manual hall booking processes, including difficulties in availability management, pricing transparency, and booking coordination. The proposed system provides a centralized digital platform that enables customers to explore halls, calculate event costs, submit reservations, and complete payments efficiently.

Throughout the development process, the main objectives of the project were successfully achieved. The final system includes secure user authentication, hall and package management, a multi-step booking workflow, PayHere payment integration, automated email notifications, customer dashboards, and real-time administrative monitoring.

The system was successfully implemented and deployed using Firebase Hosting with Cloud Firestore as the real-time database platform. The project demonstrated the importance of effective NoSQL database design, especially in areas such as preventing duplicate bookings through custom validation logic. Additionally, the use of Firestore real-time listeners improved the overall user experience by providing instant updates without requiring manual page refreshes.

Overall, HallCamilla provides a reliable and scalable solution for modernizing the hall booking process while improving operational efficiency for administrators and convenience for customers.

7.2 Future Work

Although HallCamilla successfully fulfills its current objectives, several improvements can be introduced in future development phases:

- **Live Payment Gateway Integration**

The current PayHere integration operates using Sandbox mode. Future improvements can include migrating to a production payment environment after completing official merchant registration, allowing real financial transactions.

- **Dedicated Email Service Integration**

The current EmailJS notification system can be enhanced by integrating a dedicated backend email service such as SendGrid or AWS SES to improve security, reliability, and scalability for larger communication volumes.

- **SMS Notification System**

An SMS notification feature can be introduced to provide customers with instant booking confirmations, approval updates, and payment reminders through mobile messages.

- **Advanced Payment Management**

Future versions can support flexible payment options, including customized installment plans, automated refund calculations, and improved cancellation payment workflows.

- **Mobile Application Development**

A dedicated mobile application can be developed for customers and administrators to provide easier access to booking management, notifications, and real-time updates.

- **Automated Testing Framework**

Future development can include automated testing tools such as Jest or Cypress to reduce manual testing efforts and improve system reliability as additional features are introduced.

These enhancements would extend HallCamilla from its current functional version into a more comprehensive hotel and event management platform capable of supporting larger-scale business operations.

APPENDICES

Appendix A: List of Abbreviations

Table A . 1 : Abbreviations Used in This Report

Abbreviation	Meaning
API	Application Programming Interface
CSS	Cascading Style Sheets
ERD	Entity Relationship Diagram
FK	Foreign Key
HNDIT	Higher National Diploma in Information Technology
IEEE	Institute of Electrical and Electronics Engineers
MVP	Minimum Viable Product
NoSQL	Not Only SQL
PDF	Portable Document Format
PK	Primary Key
SaaS	Software as a Service
SPA	Single Page Application
SRS	Software Requirements Specification
UAT	User Acceptance Testing
UI	User Interface
UX	User Experience

REFERENCES

- EmailJS. (n.d.). EmailJS Documentation. Retrieved July 14, 2026, from <https://www.emailjs.com/docs/>
- Git Project. (n.d.). Git Documentation. Retrieved July 14, 2026, from <https://git-scm.com/doc>
- Google. (n.d.). Firebase Documentation: Cloud Firestore and Firebase Authentication. Retrieved July 14, 2026, from <https://firebase.google.com/docs>
- Microsoft. (n.d.). Visual Studio Code Documentation. Retrieved July 14, 2026, from <https://code.visualstudio.com/docs>
- Node.js Foundation. (n.d.). Node.js Documentation. Retrieved July 14, 2026, from <https://nodejs.org/docs/latest/api/>
- npm, Inc. (n.d.). npm Documentation. Retrieved July 14, 2026, from <https://docs.npmjs.com/>
- Parallax. (n.d.). jsPDF Documentation. Retrieved July 14, 2026, from <https://parall.ax/products/jspdf>
- PayHere. (n.d.). PayHere Developer Documentation. Retrieved July 14, 2026, from <https://support.payhere.lk/api-&-plugins>
- React Core Team. (n.d.). React Documentation. Retrieved July 14, 2026, from <https://react.dev/>
- React Router Team. (n.d.). React Router Documentation. Retrieved July 14, 2026, from <https://reactrouter.com/>
- Tailwind Labs. (n.d.). Tailwind CSS Documentation. Retrieved July 14, 2026, from <https://tailwindcss.com/docs>
- TypeScript Team. (n.d.). TypeScript Documentation. Retrieved July 14, 2026, from <https://www.typescriptlang.org/docs/>
- Vite Team. (n.d.). Vite Documentation. Retrieved July 14, 2026, from <https://vite.dev/>